

ON VISUALIZING THE GEOMETRY OF OPTIMIZATION

Tai Wai Hu

Leinweber Center for Theoretical Physics
 Department of Physics
 University of Chicago
 Chicago, IL 60637, USA
 twhu@uchicago.edu

ABSTRACT

Neural networks with the same architecture, trained on the same data to comparable loss, can nonetheless differ depending on the optimization algorithm used. This dependence is attributed to the optimizer’s *implicit bias*: among the many parameter configurations that fit the training data, each optimizer converges preferentially to particular ones. In this project, we ask whether these differences are detectable in the networks’ internal representations and give an answer through the lens of representational similarity analysis (RSA), a technique from computational neuroscience which compares representations through the pairwise dissimilarities they induce between input. We find that the choice of optimizer leads to measurable differences in neural representations, even after accounting for variation due to random initialization and data ordering. On the theoretical side, we extend the characterization of optimizer implicit bias to Adam-atan2 and provide a satisfying explanation for our results.

1 INTRODUCTION

1.1 MOTIVATION

What do vision models “see”? In recent years, vision transformers (ViT) and diffusion models have largely replaced CNNs and GANs in vision-related tasks, while the growth of both the quantity and quality of image data combined with multi-modal techniques like contrastive language-image pretraining (CLIP) or masked image modeling (MIM) have enabled superior performance across vision benchmarks. One reason for this development is the discovery that vision models generalize well by learning features helpful for downstream tasks and that feature learning can largely be self-supervised.

Our focus is to gain a better understanding of the relationship between a model and its features. What is clear is that the information of learned features must be stored in the model weights, but what is less obvious is how this information is distributed. Early papers (Olah et al., 2017) in what is now known as interpretability showed that features are (largely) in correspondence with the weights themselves and form hierarchal structures called circuits (Olah et al., 2020). That this is the case is a reflection of deep “linear” structure embedded within highly-nonlinear functions (see Millidge (2023) and references therein).

It’s worth pointing out how one might reverse-engineer the features since it hints at how to think about their relationship to neurons. We focus on CNNs for the purpose of this project, but the success of related techniques (Alain & Bengio, 2016; Huben et al., 2024) on larger, transformer-based models (Templeton et al., 2024) demonstrates the robustness of this perspective. The first step to extracting the feature of a given channel is to run the forward pass of an image of random noise through the model. The output of a channel is called its activation on the input. One then maximizes the channel activation by performing gradient ascent on the input image. Done correctly (in a regularized way so as to suppress high-frequency noise), the final input image is a strange dream-like picture (Olah et al., 2017) reflecting the platonic feature associated with the channel.

One way to view a neural network is as a parameterized function with universal approximation properties. For this project we adopt a different view where a neural network is defined by its action

on inputs. This is a more useful characterization from a practical standpoint, since we often only care about the output of a neural network rather than the specific values of its weights. Defining a neural network by how it acts on data is also mathematically natural.¹ Taking this view seriously, we can characterize neural networks purely by their output—or better, by leveraging the topology of feed-forward architectures—the output/activations of their layers.

A natural question is to ask how two neural networks compare. The answer from the functional viewpoint is to use a distance measure $\|\cdot\| : \Theta \rightarrow \mathbb{R}_+$ on the space of parameters, where Θ is a singular analytic space/variety (Hoogland, 2023) given by quotienting the non-singular parameter space by the group of symmetries. This quotient arises because permutation symmetries on weight space can give rise to different networks with otherwise identical functional outputs. For simple architectures, it is possible (Ainsworth et al., 2023) to “resolve” the singularity in a way that preserves the geometry of the generalization basin, but this approach seems difficult to scale to larger architectures with more complicated symmetries. By contrast, the output y of neurons always live in a vector space $\mathcal{Y}$, and the symmetries of y are at most a subgroup G of $\text{GL}(\dim \mathcal{Y}, \mathbb{R})$. An analysis of activations, rather than of weights, therefore benefits from scalability and being able to leverage the full theory of linear algebra. Indeed, one way (Williams et al., 2021) to view representational similarity analysis (RSA) is as a science of constructing insightful G -equivariant metrics on $\mathcal{Y}$ with different choices (Klabunde et al., 2025) of G .

In this project we visualize the relationship between the space of trained models and the way in which they are optimized during training through the lens of representational similarity. We are interested especially about the dynamics of training and their relation to the final trained model. A number of similarity measures have been previously proposed to answer this question, including SVCCA (Raghu et al., 2017), PWCCA (Morcos et al., 2018), and CKA (Kornblith et al., 2019). These metrics have been used to study the stability of representations (Morcos et al., 2018), the behavior with scaling layer width, as well as comparing representations learned across different datasets (Kornblith et al., 2019). In this paper we extend these studies to the dynamics of convolutional neural networks in-training and how learned representations can reveal the implicit bias of the optimizer.

Our theoretical contributions are an extension the proof of Theorem 1.1 of Xie & Li (2024) to characterize the limit points of Adam-atan2 and GD with Nesterov/look-ahead momentum in the full batch setting, and our empirical contributions are summarized in Section 1.2. For each point, we present empirical data and theoretical justification for our conclusions. Studies were conducted using a VGG-style convolutional autoencoder trained to optimize ℓ_2 -reconstruction loss on the CIFAR-10 dataset (for more details on the experimental setup see Section 3). The model was chosen for its simplicity and the objective was chosen for its bias for rich feature learning: classifier models are known to suffer from spurious correlation and can produce misleading features, degrading the quality of our analysis. By contrast, the task of reconstruction forces the model to learn features with varying scale and complexity.

1.2 SUMMARY OF RESULTS

- **Deeper Representations Become More Varied With Training** (Section 4.2): the RSA similarity scores for deeper layers were smaller compared to shallower ones.
- **Choice of Optimizer Affects How Uniformly Representations Converge** (Section 4.3): the RSA similarity scores converged to their final values at different rates for different optimizers. SGD showed a “bottom-up” pattern where earlier layers converge almost immediately followed by deeper ones, whereas adaptive optimizers led to more uniform patterns across layer.
- **Choice of Optimizer Affects the Final Location of Models in Representation Space** (Section 4.4): using RSA dissimilarity as a non-metric dissimilarity score, low-dimensional visualizations show clustering of models based on optimizer used, reflecting their difference in implicit bias.

¹Cf. the Yoneda lemma, https://en.wikipedia.org/wiki/Yoneda_lemma.

1.3 COMMENTS

During the documentation of this project (September 2026), the author learned about the paper Zhang et al. (2026) which explores the relationship between the implicit bias of adaptive momentum optimizers and generalization of vision/language models from the functional viewpoint in great analytic depth, with a particular focus on mode connectivity in the low-loss basin. Interested readers are encouraged to take a look.

1.4 ACKNOWLEDGMENTS

This project benefitted from discussions with Frederic Koehler, Matt Walter, and especially Karen Livescu. Thanks also to Zhijian Ni for helpful comments on a preliminary project proposal and to TTIC for the valuable compute used for the experiments in this project.

2 BACKGROUND

2.1 NEURAL REPRESENTATIONS AND ACTIVATION VECTORS

Denote by $f^{(\ell)}$ the convolutional layer ℓ of our model, consisting of $c^{(\ell)}$ channels of size $(h^{(\ell)}, w^{(\ell)})$. The *neural pre-activation* of layer ℓ given an input image z is the output $z^{(\ell)}$ of $f^{(\ell)}$, a tensor of shape $(c^{(\ell)}, h^{(\ell)}, w^{(\ell)})$. For our case of convolutional autoencoder these will be the output of `Conv2d`. If $\sigma^{(\ell)}$ is the activation function at layer ℓ , the *neural activation* is the output $\sigma^{(\ell)}(z^{(\ell)})$ of $\sigma^{(\ell)} \circ f^{(\ell)}$, a tensor of the same shape, in our case given by the output of the element-wise application of `ReLU`.

Let $Z = \{z_i\}_{i=1}^m$ be an i.i.d. sample drawn from the test distribution $\mathcal{D}_{\text{test}}$. We call the *pre-activation vector at layer ℓ* the tensor

$$Z^{(\ell)} = [z_1^{(\ell)}, z_2^{(\ell)}, \dots, z_m^{(\ell)}]$$

of shape $(m, c^{(\ell)}, h^{(\ell)}, w^{(\ell)})$. In this project we analyze the neural pre-activations of layers rather than their activations for the reason that they live in a genuine vector space; post-ReLU activations live in a half-space with oriented boundary. Using activations presents an issue since the oriented boundary is basis-dependent when the RSA scores we are using are basis-independent.

2.2 REPRESENTATIONAL SIMILARITY MEASURES

Let f, g be two convolutional layers (which may belong to the same or different models). The *representational similarity* between f and g is defined to be

$$\text{sim}(f, g) := \rho(Z_f, Z_g)$$

where ρ is a *similarity measure* on the space of neural (pre)-activations and Z_f, Z_g are the neural pre-activations of layers f, g . This project focused on two similarity measures ρ : Projected Weighted Canonical Correlation Analysis (PWCCA) and Centered Kernel Alignment (CKA). Note that neither measure is an honest metric on $V^{(\ell)}$, which makes the interpretation of $\text{sim}(f, g)$ as a distance on the space of layers unfaithful.

2.2.1 PROJECTED WEIGHTED CANONICAL CORRELATION ANALYSIS (PWCCA)

PWCCA is a technique based on Canonical Correlation Analysis (CCA) used to analyze the correlation between two sets of multi-dimensional data $X \in \mathbb{R}^{m \times d_1}$, $Y \in \mathbb{R}^{m \times d_2}$. CCA iteratively looks for the linear combinations of elements from each set of data which are maximally correlated and returns that maximum correlation coefficients, such that subsequent canonical directions are uncorrelated with previous ones. In the traditional CCA algorithm, the CCA score is simply the average of correlation coefficients. A limitation of CCA is that it is easily drowned out by noise: CCA assigns a high correlation coefficient if it finds a direction that aligns pure noise in X with pure noise in Y , even if that direction only accounts for a tiny fraction of the variance in the data. PWCCA overcomes this limitation by assigning a “responsibility weight” α_i to each canonical direction based on how much of the data’s covariance that direction actually explains, so that more important directions are weighted more heavily. Another approach (SVCCA) prunes the data prior to performing CCA to only the directions which explain e.g. 99% of the total variance in the data.

Concretely, PWCCA does the following. For $1 \leq i \leq k$, the i -th canonical correlation coefficient ρ_i between X and Y is given by

$$\rho_i(X, Y) = \max_{v_i, w_i} \frac{\langle Xv_i, Yw_i \rangle}{\|Xv_i\| \|Yw_i\|}$$

subject to the constraint that $\langle Xv_i, Xv_j \rangle = 0$ and $\langle Yw_i, Yw_j \rangle = 0$ for all $j < i$ (subsequent canonical directions are uncorrelated with previous ones). PWCCA then computes a weighted sum of canonical coefficients

$$\rho_{\text{PWCCA}}(X, Y) = \frac{\sum_{i=1}^k \alpha_i \rho_i}{\sum_{i=1}^k \alpha_i}, \quad \text{where } \alpha_i = \sum_j |\langle h_i, u_j \rangle|$$

is the ‘‘responsibility weight’’ of direction i for the activations X . Here u_j is the j -th column of X and $h_i = Xv_i$ is the projection of X onto the i -th canonical direction. Note that PWCCA is not symmetric in X, Y . The solution to the constrained optimization problem for the ρ_i is given by Algorithms 1 and 2.

Algorithm 1 Projection Weighted Canonical Correlation Analysis (PWCCA)

Require: Activation matrices $X \in \mathbb{R}^{m \times d_1}$ and $Y \in \mathbb{R}^{m \times d_2}$

Ensure: PWCCA similarity score s

1: $X \leftarrow X - \text{mean}(X, \text{axis} = 0)$

2: $Y \leftarrow Y - \text{mean}(Y, \text{axis} = 0)$

3: $W_X, W_Y, \rho \leftarrow \text{CCA}(X, Y)$

4: $H \leftarrow XW_X$

5: **for** $i = 1$ **to** $\text{len}(\rho)$ **do**

6: $\tilde{\alpha}_i \leftarrow \sum_{j=1}^{d_1} |\langle h_i, u_j \rangle|$

7: **end for**

8: **for** $i = 1$ **to** $\text{len}(\rho)$ **do**

9: $\alpha_i \leftarrow \tilde{\alpha}_i / \sum_{k=1}^{\text{len}(\rho)} \tilde{\alpha}_k$

10: **end for**

11: $s \leftarrow \sum_{i=1}^{\text{len}(\rho)} \alpha_i \rho_i$

12: **return** s

Algorithm 2 Canonical Correlation Analysis (CCA)

Require: Centered matrices $X \in \mathbb{R}^{m \times d_1}$, $Y \in \mathbb{R}^{m \times d_2}$, regularizer ϵ

Ensure: Canonical weights W_X, W_Y and correlations ρ

1: $\Sigma_{XX} \leftarrow \frac{1}{m-1} X^\top X + \epsilon I$

2: $\Sigma_{YY} \leftarrow \frac{1}{m-1} Y^\top Y + \epsilon I$

3: $\Sigma_{XY} \leftarrow \frac{1}{m-1} X^\top Y$

4: $K \leftarrow \Sigma_{XX}^{-1/2} \Sigma_{XY} \Sigma_{YY}^{-1/2}$

5: $U, S, V^\top \leftarrow \text{SVD}(K)$

6: $W_X \leftarrow \Sigma_{XX}^{-1/2} U$

7: $W_Y \leftarrow \Sigma_{YY}^{-1/2} V$

8: $\rho \leftarrow \text{diag}(S)$

9: **return** W_X, W_Y, ρ

▷ canonical correlations ρ are put in descending order

2.2.2 CENTERED KERNEL ALIGNMENT (CKA)

Another representational similarity measure we use in our analysis is Centered Kernel Alignment (CKA). CKA is motivated by the observation that, for centered matrices X, Y ,

$$\|\text{Cov}(X^\top, Y^\top)\|_F^2 = \frac{1}{(m-1)^2} \text{Tr}(XX^\top YY^\top). \quad (1)$$

To unpack this equation, note that the trace on the right is equal to $\langle \text{vec}(XX^\top), \text{vec}(YY^\top) \rangle = \|Y^\top X\|_F^2$. The quantities $XX^\top, YY^\top$ are the Gram matrices of X and Y and represent the similarity between representations generated by examples within X and Y . More explicitly, $(XX^\top)_{ij}$ is the inner product of the feature vector of the i -th and j -th examples. So $\|Y^\top X\|_F^2$, equal to the inner product of flattened Gram matrices, computes a “similarity of similarities.” Kornblith et al. (2019) generalizes Equation 1 to inner products on Reproducing Kernel Hilbert Spaces (RKHS), where the Frobenius norm $\|\cdot\|_F^2$ of the cross-covariance matrix is replaced by the squared Hilbert–Schmidt norm of an arbitrary kernel operator, of which covariance is an example ($K(X, Y) = X^\top Y$).

Why might one want to do this? Suppose we are interested in comparing two probability distributions P, Q (in the case of interest P, Q will be the joint and product of marginal distributions of pre-activations, which are themselves functional transform of sample space $\mathcal{D}$). A natural way to go about this is to compare their means, variances, or higher moments. If P, Q are both bounded and have compact support, then $P = Q$ if and only if all their higher moments agree. This is however an impossible condition to check! A better approach is to map P, Q into an infinite-dimensional feature space and utilize the theory of Reproducing Kernel Hilbert Spaces (RKHS) to give a definite answer. We will now unpack this following these notes (Schrab, 2025), but the details of RKHS will not play an important role in the main analysis and can be skipped.

The only operation on P, Q that can be estimated cheaply is expectation,

$$\mathbb{E}_P(f(X)) = \int dP f \approx \frac{1}{m} \sum_{i=1}^m f(X_i).$$

A natural strategy to compare P, Q is to compute the differences in expectation they assign to a class of test functions

$$D_{\mathcal{F}}(P, Q) := \sup_{f \in \mathcal{F}} (\mathbb{E}_P f(X) - \mathbb{E}_Q f(X)).$$

For example, if $\mathcal{F}$ is the set of all linear functions ax with $a \leq 1$, then $D(P, Q) \equiv \|\mathbb{E}_P X - \mathbb{E}_Q X\|$. This is simple to compute but cannot distinguish distributions which have e.g. different variances. In the other extreme, if $\mathcal{F}$ is the set of all measurable functions with norm $\|f\| \leq 1$, then $D(P, Q) \propto \delta(P, Q)$ is the variational distance: $\delta(P, Q) = 0$ if and only if $P = Q$. However, $D(P, Q)$ does not have a closed-form expression.

The class $\mathcal{F}$ of functions should be large enough to separate distributions, yet structured enough so that the supremum has a closed-form solution. One good choice of $\mathcal{F}$ is the set of all functions f in an RKHS $\mathcal{H}$ with norm $\|f\|_{\mathcal{H}} \leq 1$. This is the definition of Maximum Mean Discrepancy (MMD):

$$\text{MMD}(P, Q) := \sup_{\|f\|_{\mathcal{H}} \leq 1} (\mathbb{E}_P f(X) - \mathbb{E}_Q f(X)).$$

We will see that MMD is expressive: if the reproducing kernel of $\mathcal{H}$ is *characteristic*, then $\text{MMD}(P, Q) = 0$ implies $P = Q$. MMD is also computable: for a given RKHS with reproducing kernel K , MMD has a cheap plug-in estimate.

Consider the Hilbert space $\mathcal{H}$ of real functions on $\mathcal{X}$, where the point evaluation function $\delta_x : \mathcal{H} \rightarrow \mathbb{R}$ which sends $f \mapsto f(x)$ is bounded for each $x \in \mathcal{X}$. Riesz’s representation theorem guarantees the existence of a unique $k_x \in \mathcal{H}$ with $f(x) = \langle f, k_x \rangle$. Define the *kernel* $k(x, y) := k_x(y) = \langle k_x, k_y \rangle$. It is manifestly symmetric and positive semidefinite. It is also useful to define the *feature map* $\varphi : \mathcal{X} \rightarrow \mathcal{H}$ by $\varphi(x) = k_x = k(x, \cdot)$.

The *mean embedding* of a distribution P is defined to be the expectation of the feature map over P ,

$$\mu_P := \mathbb{E}_{X \sim P}[k(X, \cdot)] \in \mathcal{H}.$$

The mean embedding μ_P is the Riesz dual of the functional $f \mapsto \mathbb{E}_P f(X)$,

$$\mathbb{E}_P f(X) = \langle f, \mu_P \rangle.$$

For example, take $\mathcal{X} = \mathbb{R}$ and $k(x, y) = (1 + xy)^2 = 1 + 2xy + x^2 y^2$. Then the feature map is $\varphi(x) = [1, \sqrt{2}x, x^2] \in \mathbb{R}^3$ and $\mu_P = [1, \sqrt{2}\mathbb{E}_P X, \mathbb{E}_P X^2]$. For richer kernels μ_P encodes richer information about P . A kernel is called *characteristic* if $P \mapsto \mu_P$ is injective.

Using this, we can now rewrite Maximum Mean Discrepancy (MMD) as

$$\text{MMD}(P, Q) = \sup_{\|f\|_{\mathcal{H}} \leq 1} \langle f, \mu_P - \mu_Q \rangle = \|\mu_P - \mu_Q\|_{\mathcal{H}}$$

where in the second step we have used Cauchy–Schwarz, with equality when $f^* = (\mu_P - \mu_Q)/\|\mu_P - \mu_Q\|_{\mathcal{H}}$. Expanding squared MMD,

$$\text{MMD}^2(P, Q) = \langle \mu_P, \mu_P \rangle - 2\langle \mu_P, \mu_Q \rangle + \langle \mu_Q, \mu_Q \rangle \quad (2)$$

we see that each term is an expectation of the kernel k . For example, with $X \sim P$ and $Y \sim Q$,

$$\langle \mu_P, \mu_Q \rangle = \mathbb{E}_{X \sim P} \mu_Q(X) = \mathbb{E}_{X \sim P} \mathbb{E}_{Y \sim Q} k(X, Y).$$

It follows that

$$\text{MMD}^2(P, Q) = \mathbb{E}k(X, X') - 2\mathbb{E}k(X, Y) + \mathbb{E}k(Y, Y')$$

where $X, X' \sim P$ and $Y, Y' \sim Q$ are all mutually independent. In practice this latter assumption is violated on the diagonal of $k(X, X')$ and $k(Y, Y')$ since we work with fixed samples $\{X_i\}_{i=1}^m, \{Y_i\}_{i=1}^m$. We will comment on this further below.

The Hilbert–Schmidt Independence Criterion (HSIC) test the similarity between two distributions P_X, P_Y on $\mathcal{X}$ and $\mathcal{Y}$ by computing the MMD between the joint distribution P_{XY} and product of marginals $P_X \otimes P_Y$. It is defined by

$$\text{HSIC}^2 := \text{MMD}_{k_{\mathcal{X}} \otimes k_{\mathcal{Y}}}^2(P_{XY}, P_X \otimes P_Y) = \|\mu_{P_{XY}} - \mu_{P_X} \otimes \mu_{P_Y}\|^2$$

Note that the last expression is the (Hilbert–Schmidt) norm squared $\|\mathcal{C}_{XY}\|_{\text{HS}}^2$ of the cross-covariance operator

$$\mathcal{C}_{XY} = \mu_{P_{XY}} - \mu_{P_X} \otimes \mu_{P_Y} = \mathbb{E}[(\varphi(X) - \mu_{P_X}) \otimes (\psi(Y) - \mu_{P_Y})]$$

It is instructive to understand what $\mathcal{C}_{XY}$ is. The algebraic product of Hilbert Spaces $\mathcal{H}_{\mathcal{X}} \otimes \mathcal{H}_{\mathcal{Y}}$ is isomorphic to the space of Hilbert–Schmidt operators $\mathcal{H}_{\mathcal{Y}} \rightarrow \mathcal{H}_{\mathcal{X}}$, and under this isomorphism $a \otimes b \mapsto (g \mapsto \langle b, g \rangle a)$. Therefore, for any functions $f \in \mathcal{H}_{\mathcal{X}}, g \in \mathcal{H}_{\mathcal{Y}}$:

$$\begin{aligned} \langle f, \mathcal{C}_{XY} g \rangle_{\mathcal{H}_{\mathcal{X}}} &= \mathbb{E} \langle f, (\varphi(X) - \mu_{P_X}) \otimes (\psi(Y) - \mu_{P_Y}) g \rangle_{\mathcal{H}_{\mathcal{X}}} \\ &= \mathbb{E} [\langle f, \varphi(X) - \mu_{P_X} \rangle_{\mathcal{H}_{\mathcal{X}}} \langle g, \psi(Y) - \mu_{P_Y} \rangle_{\mathcal{H}_{\mathcal{Y}}}] \\ &= \mathbb{E} [(f(X) - \mathbb{E}f(X)) (g(Y) - \mathbb{E}g(Y))] \end{aligned}$$

is just the covariance of the features $f(X)$ and $g(Y)$, for any choice of f, g . Therefore, HSIC can be thought of as the Hilbert–Schmidt norm of the cross-covariance operator; this is a direct generalization of the LHS of Equation 1.

Expanding Equation 2 with the product kernel gives

$$\begin{aligned} \text{HSIC}^2 &= \mathbb{E}[k^{\mathcal{X}}(X, X')k^{\mathcal{Y}}(Y, Y')] - 2\mathbb{E}_{(X, Y)}[\mathbb{E}_{X'}k^{\mathcal{X}}(X, X')\mathbb{E}_{Y'}k^{\mathcal{Y}}(Y, Y')] \\ &\quad + \mathbb{E}[k^{\mathcal{X}}(X, X')]\mathbb{E}[k^{\mathcal{Y}}(Y, Y')] \end{aligned}$$

Suppose we have m samples (X_i, Y_i) drawn from the two distributions $\mathcal{X} \times \mathcal{Y}$. To estimate HSIC, we replace all distributions with their empirical counterparts:

$$\hat{P}_{XY} = \frac{1}{m} \sum_{i=1}^m \delta_{(X_i, Y_i)}, \quad \hat{P}_X = \frac{1}{m} \sum_{i=1}^m \delta_{X_i}, \quad \hat{P}_Y = \frac{1}{m} \sum_{i=1}^m \delta_{Y_i}.$$

It follows that, with $K_{ij} = k^{\mathcal{X}}(x_i, x_j)$ and $L_{ij} = k^{\mathcal{Y}}(y_i, y_j)$

$$\begin{aligned} \mathbb{E}[k^{\mathcal{X}}(X, X')k^{\mathcal{Y}}(Y, Y')] &\rightarrow \frac{1}{m^2} \sum_{i,j=1}^m K_{ij}L_{ij} = \frac{1}{m^2} \text{Tr}(KL) \\ \mathbb{E}_{(X, Y)}[\mathbb{E}_{X'}k^{\mathcal{X}}(X, X')\mathbb{E}_{Y'}k^{\mathcal{Y}}(Y, Y')] &\rightarrow \frac{1}{m} \sum_{i=1}^m \left(\frac{1}{m} \sum_{j=1}^m K_{ij} \right) \left(\frac{1}{m} \sum_{r=1}^m L_{ir} \right) = \frac{1}{m^3} \mathbf{1}^\top K L \mathbf{1} \\ \mathbb{E}[k^{\mathcal{X}}(X, X')]\mathbb{E}[k^{\mathcal{Y}}(Y, Y')] &\rightarrow \left(\frac{1}{m^2} \sum_{i,j=1}^m K_{ij} \right) \left(\frac{1}{m^2} \sum_{i,j=1}^m L_{ij} \right) = \left(\frac{1}{m^2} \mathbf{1}^\top K \mathbf{1} \right) \left(\frac{1}{m^2} \mathbf{1}^\top L \mathbf{1} \right) \end{aligned}$$

where we have used that K, L are symmetric and $\sum_j K_{ij} = (K\mathbf{1})_i$ (sim. for L). Combining these,

$$\text{HSIC}(K, L) = \frac{1}{m^2} \text{Tr}(KL) - \frac{2}{m^3} \underbrace{\mathbf{1}^\top K L \mathbf{1}}_{\text{tr}(K\mathbf{1}\mathbf{1}^\top L) + \text{tr}(KL\mathbf{1}\mathbf{1}^\top)} + \left(\frac{1}{m^2} \mathbf{1}^\top K \mathbf{1} \right) \left(\frac{1}{m^2} \mathbf{1}^\top L \mathbf{1} \right) = \frac{1}{m^2} \text{Tr}(KHLH)$$

where $H = \mathbb{I}_m - \frac{1}{m} \mathbf{1}\mathbf{1}^\top$ is the centering matrix. This is the empirical estimator of HSIC in Kornblith et al. (2019) up to normalization which drops out of CKA, which is simply defined to be normalized HSIC:

$$\rho_{\text{CKA}}(X, Y) = \frac{\text{HSIC}(K, L)}{\sqrt{\text{HSIC}(K, K) \text{HSIC}(L, L)}}$$

This estimator converges to the population value at a rate of $O(m^{-1/2})$. However, it is not unbiased: the computation of MMD² requires that X, X' be different i.i.d. samples drawn from the population distribution, but here the Gram matrices K, L are computed using just one draw for both. A true unbiased estimate is provided by Song et al. (2012):

$$\text{HSIC}_{\text{unbiased}}(K, L) = \frac{1}{m(m-3)} \left[\text{Tr}(\bar{K}\bar{L}) + \frac{\mathbf{1}^\top \bar{K} \mathbf{1} \mathbf{1}^\top \bar{L} \mathbf{1}}{(m-1)(m-2)} - \frac{2}{m-2} \mathbf{1}^\top \bar{K} \bar{L} \mathbf{1} \right]$$

where $\bar{K}, \bar{L}$ are the kernel matrices K, L with diagonal entries set to zero. This differs from the biased estimator by terms of order- m^{-2} , and is negligible for our applications (we use $m = 128$). For ease of computation, we follow the conventions of Kornblith et al. and use the biased estimator.

HSIC allows for generalizations to any kernel of choice, and Kornblith et al. also tried Gaussian RBF, an universal kernel. They showed that linear and Gaussian RBF kernels gave similar results and therefore default to the linear kernel, in which case the plug-in estimator reduces to Equation 1 with $K = XX^\top$ and $L = YY^\top$ centered. The procedure is summarized in Algorithm 3.

Algorithm 3 Centered Kernel Alignment (CKA)

Require: Activation matrices $X \in \mathbb{R}^{m \times d_1}$ and $Y \in \mathbb{R}^{m \times d_2}$

Ensure: CKA similarity score s

- 1: $K^X \leftarrow XX^\top$
 - 2: $K^Y \leftarrow YY^\top$
 - 3: $H \leftarrow \mathbb{I}_m - \frac{1}{m} \mathbf{1}\mathbf{1}^\top$
 - 4: $\mu_{XX} \leftarrow \frac{1}{(m-1)^2} \text{Tr}(K^X H K^X H)$
 - 5: $\mu_{XY} \leftarrow \frac{1}{(m-1)^2} \text{Tr}(K^X H K^Y H)$
 - 6: $\mu_{YY} \leftarrow \frac{1}{(m-1)^2} \text{Tr}(K^Y H K^Y H)$
 - 7: $s \leftarrow \mu_{XY} / \sqrt{\mu_{XX} \mu_{YY}}$
 - 8: **return** s
-

2.3 AGGREGATE MODEL-TO-MODEL METRICS

The similarity measures ρ_{PWCCA} and ρ_{CKA} allow for comparisons between two convolutional layers. One way to extend to model-wise similarity is to perform RSA on a concatenation of pre-activations over layers. This is however very computationally expensive. For an L -layer network, the complexity for computing the activation vector and Gram matrix are $O(L)$ and $O(L^2)$, respectively, and the overall complexity for PWCCA is $O(mL^2)$, where m is the number of images sampled for the activation vector.

We introduce a cheaper surrogate algorithm for computing model-wise RSA scores in the special case that two models in question are identical in architecture. We average over the layer-wise dissimilarities of the networks, a metric which we call *mean diagonal dissimilarity* μ_0 . Dissimilarity d here is defined to be $d = 1 - s$, where s is the similarity score.

However, there is the possibility that the same features are learned slightly earlier/later in the model, depending on the initialization and data randomization, so that layer ℓ in model A aligns most closely with layer ℓ' in model B with $|\ell' - \ell| \leq b$. To account for this, we introduce a second algorithm

which we call *banded diagonal dissimilarity* μ_b , which aggregates layer-wise similarities between layers that are offset by up to b positions in their models and select the minimum (most similar) value within that band of size $2b + 1$. Note that banded diagonal dissimilarity reduces to mean diagonal dissimilarity when $b = 0$.

To be concrete about this, let X_i, Y_i be the neural pre-activations of layer i in the two models and denote by $d_{i,j} = 1 - \rho(X_i, Y_j)$ the dissimilarity between layers X_i and Y_j . Then mean diagonal dissimilarity is defined to be

$$\mu_0 = \frac{1}{L} \sum_{i=1}^L d_{i,i},$$

and banded diagonal dissimilarity with band b is defined to be

$$\mu_b = \frac{1}{L} \sum_{i=1}^L \min_{\substack{j \\ |i-j| \leq b}} d_{i,j}.$$

2.4 OPTIMIZERS

Having fixed the architecture and dataset used for training, the next most important factor which determines learned features is the optimization algorithm, which includes a choice of optimizer, its hyperparameters, and learning schedule. Optimization affects the learned features due to its implicit bias, the phenomenon of gradient-based algorithms to select specific global minimum solutions from many possibilities.

This study compares SGD, AdamW, and a stable variant of AdamW called Adam-atan2. Table 1 summarizes their differences in implicit bias.

	SGD	Adaptive Momentum
Implicit Bias on Separable Data (no Weight Decay)	Max ℓ_2 -Margin or Direction (Soudry et al., 2018)	Max ℓ_∞ -Margin or Direction (Zhang et al., 2024)
Limit Point KKT Characterization (with Weight Decay)	ℓ_2 -norm Constrained Optimizer	ℓ_∞ -norm Constrained Optimizer (Xie & Li, 2024)

Table 1: Implicit bias of SGD versus adaptive momentum methods.

An interesting extension of this study would be to incorporate modern optimizers such as Lion (implicit bias on ℓ_∞ -norm) and Muon (implicit bias on spectral norm), see e.g. Sfyraki & Wang (2025). As a dynamical process, optimization algorithms also inject an implicit bias during training in a data and objective-dependent way that is not studied in this project, see e.g. Xie et al. (2025).

2.4.1 ADAMW

AdamW, or Adaptive Momentum with Decoupled Weight Decay, is an adaptive momentum optimization algorithm which applies weight decay directly to the weights at every step, decoupling it from the gradient calculations (Algorithm 4).

A few comments on Algorithm 4:

1. The quantities m_t, v_t are the moving averages of the first (momentum) and second (RM-Sprop) moments of the gradient g_t .
2. Lines 7, 8 apply an important bias correction to the first and second moments m_t, v_t since early in training they are very small.
3. The weight update in line 9 separately applies weight decay $\lambda\theta_{t-1}$ from momentum $\frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}$. This distinguishes AdamW from Adam with ℓ_2 -regularization, which applies “weight de-

Algorithm 4 Adam with Decoupled Weight Decay (AdamW)

Require: Learning rate η , weight decay λ , exponential decay rates $\beta_1, \beta_2 \in [0, 1)$, constant $\epsilon > 0$, initial parameters θ_0 , objective function $L(\theta)$

Ensure: Final parameters θ_T

```

1:  $m_0 \leftarrow 0, v_0 \leftarrow 0, t \leftarrow 0$ 
2: while  $\theta_t$  not converged do
3:    $t \leftarrow t + 1$ 
4:    $g_t \leftarrow \nabla_{\theta} L(\theta_{t-1})$ 
5:    $m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) g_t$ 
6:    $v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$ 
7:    $\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$ 
8:    $\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$ 
9:    $\theta_t \leftarrow \theta_{t-1} - \eta \left( \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}} + \lambda \theta_{t-1} \right)$ 
10: end while
11: return  $\theta_t$ 

```

cay” to the gradient in line 4,

$$g_t \leftarrow \nabla_{\theta} L(\theta_{t-1}) + \lambda \theta_{t-1}.$$

The following theorem was proven in Xie & Li (2024) characterizing the implicit bias of AdamW.

Theorem 2.1 (Xie & Li, 2024, Theorem 1.1). *For any continuously differentiable objective function $L : \mathbb{R}^d \rightarrow \mathbb{R}$, $\beta_1 \leq \beta_2 < 1$, initialization x_0 and non-increasing learning rate $\{\eta_t\}_{t=1}^{\infty}$ such that $\sum_t \eta_t = \infty$, if the iterates of AdamW on L converge to some x_{∞} , then x_{∞} is a KKT point of the constrained optimization problem*

$$\min_{\|x\|_{\infty} \leq 1/\lambda} L(x).$$

If L is additionally convex, then AdamW converges to the constrained minimizer,

$$x_{\infty} \in \arg \min_{\|x\|_{\infty} \leq 1/\lambda} L(x).$$

Proof (sketch). We present a proof sketch that only proves the theorem of interest, omitting the interesting connections with Frank-Wolfe and SignGD. Interested readers are encouraged to read Xie & Li (2024). We begin by recalling the definition of a KKT point. Consider the general continuous optimization problem

$$\begin{aligned} & \min_{x \in \mathbb{R}^n} f(x) \\ & \text{subject to } g_i(x) \leq 0, \quad i = 1, \dots, m \\ & \quad \quad h_j(x) = 0, \quad j = 1, \dots, p \end{aligned}$$

where $f, g_i, h_i : \mathbb{R}^n \rightarrow \mathbb{R}$ are all continuously-differentiable functions. Define the Lagrangian

$$\mathcal{L}(x, \mu, \nu) = f(x) + \langle \mu, g(x) \rangle + \langle \nu, h(x) \rangle.$$

The vectors μ, ν are generalized Lagrange multipliers called *dual variables*.

A point $(x^*, \mu^*, \nu^*) \in \mathbb{R}^n \times \mathbb{R}^m \times \mathbb{R}^p$ is a *Karush–Kuhn–Tucker (KKT)* point if it satisfies the following conditions:

1. *Stationarity:* $0 \in \partial \mathcal{L}(x^*, \mu^*, \nu^*)$
2. *Primal Feasibility:* $g_i(x^*) \leq 0$ and $h_j(x^*) = 0$ for all i, j
3. *Dual Feasibility:* $\mu_i^* \geq 0$ for all $i = 1, \dots, m$
4. *Complementary Slackness:* $\langle \mu^*, g(x^*) \rangle = 0$.

A KKT point with positive-definite Hessian is a local minimum. Moreover, if the objective function f and the g_i are convex, and h_i are affine, then (x^*, μ^*, ν^*) being a KKT point implies that x^* is a global minimum of f .

However, not every local minimum x^* is a KKT point. In order for it to be so, the point x^* must also satisfy a regularity condition; one example is Linear Independence Constraint Qualification (LICQ), which states that the gradients of the h_i and the active g_i are linearly-independent at x^* .

It suffices to prove the first part of the theorem. Consider the Lagrangian of the constrained optimization problem:

$$\mathcal{L}(x, \mu) = L(x) + \mu \left(\|x\|_\infty - \frac{1}{\lambda} \right). \quad (3)$$

Xie and Li proved Theorem 1.1 by establishing two things:

1. **Lemma 3.8** of Xie & Li (2024) (Lemma 2.2 below): x is a KKT point of $\min_{\|x\|_\infty \leq 1/\lambda} L(x)$ if and only if

$$\|x\|_\infty \leq \frac{1}{\lambda} \quad \text{and} \quad \langle -\lambda x, \nabla L(x) \rangle = \|\nabla L(x)\|_1. \quad (4)$$

2. The limit point of AdamW satisfies the condition of Lemma 3.8.

Lemma 2.2 (Xie & Li, 2024, Lemma 3.8). x is a KKT point of $\min_{\|x\|_\infty \leq 1/\lambda} L(x)$ if and only if Equation 4 holds.

Proof. Primal feasibility for the constrained optimization problem in Equation 3 says $\|x^*\|_\infty \leq 1/\lambda$ while stationarity says

$$0 \in \partial \mathcal{L}(x^*, \mu^*) = \nabla L(x^*) + \mu^* \partial \|x^*\|_\infty,$$

or $-\nabla L(x^*) \in \mu^* \partial \|x^*\|_\infty$. Since $\mu^* \partial \|x^*\|_\infty = \{w \in \mathbb{R}^d \mid \langle w, x^* \rangle = \mu^* \|x^*\|_\infty \text{ and } \|w\|_1 \leq \mu^*\}$, stationarity implies

$$\langle -\nabla L(x^*), x^* \rangle = \mu^* \|x^*\|_\infty \text{ and } \|\nabla L(x^*)\|_1 \leq \mu^*.$$

Expand on the cases of primal feasibility:

- x^* is an interior point ($\|x^*\|_\infty < 1/\lambda$): by complementary slackness, $\mu^* = 0$, which implies $\nabla L(x^*) = 0$.
- x^* is a boundary point ($\|x^*\|_\infty = 1/\lambda$): the minimal value of μ required to satisfy stationarity is $\mu = \|\nabla L(x^*)\|_1$.

In both cases, it's true that for a KKT point (x^*, μ^*) :

$$\langle -\nabla L(x^*), x^* \rangle = \mu^* \|x^*\|_\infty = \frac{1}{\lambda} \|\nabla L(x^*)\|_1, \quad \implies \quad \langle -\lambda x^*, \nabla L(x^*) \rangle = \|\nabla L(x^*)\|_1.$$

The reverse direction follows by choosing $\mu^* = \|\nabla L(x^*)\|_1$. Assume $\|x^*\|_\infty \leq 1/\lambda$ and $\langle -\lambda x^*, \nabla L(x^*) \rangle = \|\nabla L(x^*)\|_1$. If $\nabla L(x^*) = 0$ then all four KKT conditions are vacuously true. Suppose then that $\nabla L(x^*) \neq 0$. Primal and dual feasibility are manifestly satisfied. To verify complementary slackness, we use Hölder's inequality to write

$$\|\nabla L(x^*)\|_1 = \langle -\lambda x^*, \nabla L(x^*) \rangle \leq \|\lambda x^*\|_\infty \cdot \|\nabla L(x^*)\|_1 \leq \|\nabla L(x^*)\|_1.$$

Therefore both inequalities are equalities. Dividing through by $\|\nabla L(x^*)\|_1 > 0$ gives $\|\lambda x^*\|_\infty = 1$, so

$$\mu^* \left(\|x^*\|_\infty - \frac{1}{\lambda} \right) = 0.$$

To prove stationarity note that $0 \in \nabla L(x^*) + \mu^* \partial \|x^*\|_\infty$ is equivalent to $-\frac{1}{\mu^*} \nabla L(x^*) \in \partial \|x^*\|_\infty$. By properties of dual norm subgradients, this requires

$$\left\| -\frac{1}{\mu^*} \nabla L(x^*) \right\|_1 = 1 \quad \text{and} \quad \left\langle -\frac{1}{\mu^*} \nabla L(x^*), x^* \right\rangle = \|x^*\|_\infty.$$

The first condition is true by substitution of μ^* . The second condition is true since

$$\left\langle -\frac{1}{\mu^*} \nabla L(x^*), x^* \right\rangle = \frac{\langle -\lambda x^*, \nabla L(x^*) \rangle}{\lambda \mu^*} = \frac{\|\nabla L(x^*)\|_1}{\lambda \mu^*} = \frac{1}{\lambda} = \|x^*\|_\infty. \quad \square$$

We now finish the proof sketch of Theorem 2.1 by proving that the limit point x_∞ of AdamW satisfies the condition in Equation 4. The following lemma will be of use.

Lemma 2.3 (Weighted Cesàro). *If a sequence $\{a_t\}_{t=0}^\infty$ on a normed space converges to $a_t \rightarrow a$, and another sequence $\{\eta_t\}_{t=0}^\infty$ has $\eta_t \geq 0$ and $S_T = \sum_{t \leq T} \eta_t$ has $\lim_{T \rightarrow \infty} S_T = \infty$, then*

$$\frac{1}{S_T} \sum_{t \leq T} \eta_t a_t \rightarrow a.$$

Proof. Given $\epsilon > 0$, pick t' such that $\|a_t - a\| \leq \epsilon$ for all $t > t'$. Then by triangle inequality,

$$\left\| \frac{1}{S_T} \sum_{t \leq T} \eta_t (a_t - a) \right\| \leq \frac{1}{S_T} \sum_{t \leq T} \eta_t \|a_t - a\| \leq \frac{1}{S_T} \sum_{t \leq t'} \eta_t \|a_t - a\| + \epsilon.$$

Taking the limit as $T \rightarrow \infty$, the first term on the RHS is a fixed number divided by a quantity which goes to zero. The Lemma follows by rearrangement of the LHS. $\square$

Proceeding to the main proof, let

$$\begin{aligned} g_t &= \nabla L(x_{t-1}) \\ m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \\ \Delta_t &= \frac{m_t}{\sqrt{v_t}} \quad \text{coordinate-wise} \\ x_t &= (1 - \lambda \eta_t) x_{t-1} - \eta_t \Delta_t \end{aligned}$$

with $m_0 = v_0 = 0$. Additionally, define $S_t = \sum_{t' \leq t} \eta_{t'}$ and the AdamW average update size

$$\bar{\Delta}_t = \frac{1}{S_t} \sum_{t' \leq t} \eta_{t'} \Delta_{t'}.$$

Assume that AdamW converges as $t \rightarrow \infty$, so $x_t \rightarrow x_\infty$, $g_t \rightarrow g_\infty$, and that $S_t \rightarrow \infty$.

Xie and Li's proof relies on finding an upper bound of $\bar{\Delta}_t$ as $t \rightarrow \infty$, despite the fact that individual updates Δ_t can have unbounded magnitude depending on the choice of β_1, β_2 , and that will be our goal as well.

The AdamW update rule implies $\eta_t \Delta_t = (x_{t-1} - x_t) - \lambda \eta_t x_{t-1}$. Summing up to T and dividing by S_T ,

$$\bar{\Delta}_T = \frac{1}{S_T} \sum_{t \leq T} [(x_{t-1} - x_t) - \lambda \eta_t x_{t-1}] = \frac{1}{S_T} \left[(x_0 - x_T) - \lambda \sum_{t \leq T} \eta_t x_{t-1} \right].$$

Taking the limit as $T \rightarrow \infty$ and using Weighted Cesàro on the second term, we find $\bar{\Delta}_\infty$ is well-defined and given by $-\lambda x_\infty$.

Next we turn to analyzing the components $g_{t,i}$ of the gradient as $t \rightarrow \infty$. Suppose $g_{t,i} \rightarrow g_{\infty,i} \neq 0$. Since the EMA of a convergent sequence converges to its limit, $m_{t,i} \rightarrow g_{\infty,i}$, $v_{t,i} \rightarrow g_{\infty,i}^2$. It follows that $\Delta_{t,i} \rightarrow \text{sign}(g_{\infty,i})$. By Weighted Cesàro it follows that $|\bar{\Delta}_{t,i}| \rightarrow 1$.

The trickier case is if $g_{\infty,i} = 0$. In this case, Xie and Li prove that $\bar{\Delta}_{t,i} \leq 1$ as $t \rightarrow \infty$ directly. Combined with the case when $g_{\infty,i} \neq 0$, this implies $\|\bar{\Delta}_\infty\|_\infty \leq 1$ and as a result $\|x_\infty\|_\infty \leq 1/\lambda$. Moreover,

$$\langle g_\infty, \bar{\Delta}_\infty \rangle = \sum_{i | g_{\infty,i} \neq 0} g_{\infty,i} \text{sign}(g_{\infty,i}) = \|g_\infty\|_1.$$

Substituting $\bar{\Delta}_\infty = -\lambda x_\infty$, we see that the limit point x_∞ satisfies the conditions of Lemma 2.2 and thus is a KKT point of $\min_{\|x\|_\infty \leq 1/\lambda} L(x)$.

It remains to prove that $\bar{\Delta}_{t,i} \leq 1$ as $t \rightarrow \infty$ in the case that $g_{\infty,i} \rightarrow 0$. To simplify notation we fix an index i and suppress it. It is useful to define $L_s = \log(v_s/v_1)$ for $s \geq 1$. Then $L_s \geq (s-1) \log \beta_2$ since $v_s \geq \beta_2 v_{s-1}$, and there exists s_0 such that $L_s \leq 0$ for all $s \geq s_0$ since $v_s \rightarrow 0$ asymptotically. We begin by proving the following Lemma.

Lemma 2.4. *For non-increasing learning schedule η_t and every $j \geq 1$, $T \geq j+1$,*

$$\sum_{t=j+1}^T \eta_t (L_t - L_{t-j}) \leq C_j := \eta_1 \left(s_0 \max_{s < s_0} |L_s| + \frac{j^2}{2} |\log \beta_2| \right).$$

Proof. We proceed in two cases: (1) if $T \geq 2j$ and (2) if $j+1 \leq T < 2j$. Let $s = t - j$.

For case 1, we can write the LHS as

$$\sum_{t=j+1}^T \eta_t L_t - \sum_{s=1}^{T-j} \eta_{s+j} L_s = \sum_{s=j+1}^{T-j} (\eta_s - \eta_{s+j}) L_s + \sum_{s=T-j+1}^T \eta_s L_s - \sum_{s=1}^j \eta_{s+j} L_s$$

Contributions to the first two terms run over disjoint indices s and moreover $L_s > 0$ only if $s < s_0$. Since $0 \leq \eta_s - \eta_{s+j} \leq \eta_s \leq \eta_1$, their sum is therefore bounded by $\eta_1 s_0 \max_{s < s_0} |L_s|$. The third term is bounded above by

$$- \sum_{s=1}^j \eta_{s+j} L_s \leq \eta_1 |\log \beta_2| \sum_{s=1}^j (s-1) \leq \eta_1 |\log \beta_2| \frac{j^2}{2}.$$

For case 2, we bound the LHS directly: $\sum_{t=j+1}^T \eta_t L_t \leq \eta_1 s_0 \max_{s < s_0} |L_s|$ and in this range $T - j < j$, so

$$- \sum_{s=1}^{T-j} \eta_{s+j} L_s \leq \eta_1 |\log \beta_2| \sum_{s=1}^{T-j} (s-1) \leq \eta_1 |\log \beta_2| \frac{j^2}{2}.$$

□

We now finish the proof as in Xie and Li. Expanding the recursive definitions write $m_t = (1 - \beta_1) \sum_{j=0}^t \beta_1^j g_{t-j}$, $v_t = (1 - \beta_2) \sum_{j=0}^t \beta_2^j g_{t-j}^2$. The weights $(1 - \beta_1) \beta_1^j$ sum up to at most one, so by Cauchy–Schwarz

$$m_t^2 \leq (1 - \beta_1) \sum_{j=0}^t \beta_1^j g_{t-j}^2.$$

It follows that

$$\Delta_t^2 = \frac{m_t^2}{v_t} \leq \frac{1 - \beta_1}{1 - \beta_2} \left(\frac{\sum_{j=0}^t \beta_1^j g_{t-j}^2}{v_t / (1 - \beta_2)} \right).$$

The bracketed quantity is a ratio of exponential moving averages with memory $\beta_1 \leq \beta_2$. If g^2 is constant, this ratio equals unity. If g^2 is shrinking or growing, then the ratio is < 1 or > 1 , respectively, since the most recent values dominate. Since g is unbounded, no pointwise bound for Δ_t exists. The point is that no asymptotically-vanishing sequence g_t increases on average, so a bound on $\bar{\Delta}_t$ is possible.

It is convenient to define $m_s, v_s, g_s = 0$ for $s \leq 0$. By definition $g_s^2 = (v_s - \beta_2 v_{s-1}) / (1 - \beta_2)$, so

$$(1 - \beta_2) \sum_{j \geq 0} \beta_1^j g_{t-j}^2 = \sum_{j \geq 0} \beta_1^j (v_{t-j} - \beta_2 v_{t-j-1}) = v_t - (\beta_2 - \beta_1) \sum_{j \geq 1} \beta_1^{j-1} v_{t-j}.$$

Thus,

$$v_t \Delta_t^2 = m_t^2 \leq (1 - \beta_1) \sum_{j \geq 0} \beta_1^j g_{t-j}^2 = \frac{1 - \beta_1}{1 - \beta_2} \left(v_t - (\beta_2 - \beta_1) \sum_{j \geq 1} \beta_1^{j-1} v_{t-j} \right)$$

or, dividing through by v_t ,

$$\Delta_t^2 \leq \frac{1 - \beta_1}{1 - \beta_2} \left(1 - (\beta_2 - \beta_1) \sum_{j \geq 1} \beta_1^{j-1} \frac{v_{t-j}}{v_t} \right).$$

Using the identities $(1 - \beta_1)/(1 - \beta_2) = 1 + (\beta_2 - \beta_1)/(1 - \beta_2)$ and $(1 - \beta_1) \sum_{j \geq 1} \beta_1^{j-1} = 1$,

$$\begin{aligned} \Delta_t^2 &\leq 1 + \frac{(\beta_2 - \beta_1)(1 - \beta_1)}{1 - \beta_2} \sum_{j \geq 1} \beta_1^{j-1} \left(1 - \frac{v_{t-j}}{v_t} \right) \\ &= 1 + \frac{\beta_2 - \beta_1}{1 - \beta_2} \left(\beta_1^{t-1} + (1 - \beta_1) \sum_{j=1}^{t-1} \beta_1^{j-1} \left(1 - \frac{v_{t-j}}{v_t} \right) \right) \end{aligned}$$

Since $1 - x \leq -\log x$ for $x > 0$, $1 - (v_{t-j}/v_t) \leq L_t - L_{t-j}$, and we get

$$\Delta_t^2 \leq 1 + \frac{\beta_2 - \beta_1}{1 - \beta_2} \left(\beta_1^{t-1} + (1 - \beta_1) \sum_{j=1}^{t-1} \beta_1^{j-1} (L_t - L_{t-j}) \right)$$

Now sum over t with weight η_t :

$$\sum_{t \leq T} \eta_t \Delta_t^2 \leq S_T + \frac{\beta_2 - \beta_1}{1 - \beta_2} \left(\sum_{t \leq T} \eta_t \beta_1^{t-1} + (1 - \beta_1) \sum_{t \leq T} \sum_{j=1}^{t-1} \beta_1^{j-1} \eta_t (L_t - L_{t-j}) \right)$$

The double sum is over lattice points within a triangle. The same domain has an equivalent representation $\sum_{t \leq T} \sum_{j=1}^{t-1} = \sum_{j=1}^{T-1} \sum_{t=j+1}^T$; combined with Lemma 2.4 and the bound $\sum_{t \leq T} \eta_t \beta_1^{t-1} \leq \eta_1/(1 - \beta_1)$,

$$\sum_{t \leq T} \eta_t \Delta_t^2 \leq S_T + \frac{\beta_2 - \beta_1}{1 - \beta_2} \left(\frac{\eta_1}{1 - \beta_1} + (1 - \beta_1) \sum_{j=1}^{T-1} \beta_1^{j-1} C_j \right)$$

Now $C_j = O(j^2)$, so $\sum_{j=1}^{T-1} \beta_1^{j-1} C_j \leq C \int_1^T dt \beta_1^{t-1} t^2$ and is manifestly finite for $\beta_1 < 1$. In the limit that $T \rightarrow \infty$, the integral can be evaluated exactly to give $\Gamma(3, -\log \beta_1)/(\beta_1(-\log \beta_1)^3) < \infty$. It follows that by Jensen's inequality,

$$\bar{\Delta}_T^2 \leq \frac{1}{S_T} \sum_{t \leq T} \eta_t \Delta_t^2 \leq 1 + \frac{O(1)}{S_T}.$$

Since this is true for every component i of $\bar{\Delta}_T$, it follows that for components i for which $g_{\infty,i} \rightarrow 0$, $\bar{\Delta}_{\infty,i}^2 = \lim_{T \rightarrow \infty} \bar{\Delta}_{T,i}^2 \leq 1$; in particular $|\bar{\Delta}_{\infty,i}| \leq 1$. $\square$

When training a linear classifier on separable data, Adam without weight decay converges to the ℓ_∞ -norm minimizing solution (see Zhang et al., 2024). Together with the KKT characterization of AdamW, these results suggest that adaptive momentum optimizers operate with respect to the ℓ_∞ -geometry of the loss landscape, providing a plausible explanation for its faster generalization as compared to SGD (see Section 2.4.3 and Xie et al., 2025).

2.4.2 ADAM-ATAN2

The Adam-atan2 optimizer was proposed in Everett et al. (2024) as a scale-invariant version of AdamW which gets rid of the ϵ parameter entirely. The problem with a fixed ϵ parameter is that it warps the update rule in the limit when the gradient is small. For layer width n , there is a positive exponent $g_\ell > 0$ such that the size of gradients empirically scale as n^{-g_ℓ} . Subsequently, the AdamW update rule

$$\theta_t \leftarrow \theta_{t-1} - \alpha \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda \theta_{t-1} \right)$$

is dominated by noise for wide networks. To fix this issue, Everett et al. propose the alternative update rule

$$\theta_t \leftarrow \theta_{t-1} - \alpha \left(\frac{4}{\pi} \arctan \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t}} \right) + \lambda \theta_{t-1} \right).$$

The full algorithm is given in Algorithm 5.

Algorithm 5 Adam-atan2

Require: Learning rate η , weight decay λ , exponential decay rates $\beta_1, \beta_2 \in [0, 1)$, initial parameters θ_0 , objective function $L(\theta)$

Ensure: Final parameters θ_T

```

1:  $m_0 \leftarrow 0, v_0 \leftarrow 0, t \leftarrow 0$ 
2: while  $\theta_t$  not converged do
3:    $t \leftarrow t + 1$ 
4:    $g_t \leftarrow \nabla_{\theta} L(\theta_{t-1})$ 
5:    $m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) g_t$ 
6:    $v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$ 
7:    $\hat{m}_t \leftarrow m_t / (1 - \beta_1^t)$ 
8:    $\hat{v}_t \leftarrow v_t / (1 - \beta_2^t)$ 
9:    $\theta_t \leftarrow \theta_{t-1} - \eta \left( \frac{4}{\pi} \arctan \left( \frac{\hat{m}_t}{\sqrt{\hat{v}_t}} \right) + \lambda \theta_{t-1} \right)$ 
10: end while
11: return  $\theta_t$ 

```

The similarity of this update rule to that of AdamW makes it possible to adapt the proof for AdamW (see the proof of Theorem 2.1) to uncover the implicit bias of Adam-atan2. Concretely, we prove the following Corollary.

Corollary 2.5 (Implicit Bias of Adam-atan2 in Full Batch Setting). *For any continuously differentiable objective function $L : \mathbb{R}^d \rightarrow \mathbb{R}$, $\beta_1 \leq \beta_2 < 1$, initialization x_0 and non-increasing learning rate $\{\eta_t\}_{t=1}^{\infty}$ such that $\sum_t \eta_t = \infty$, if the iterates of Adam-atan2 on L converge to some x_{∞} , then x_{∞} is a KKT point of the constrained optimization problem*

$$\min_{\|x\|_{\infty} \leq 1/\lambda} L(x).$$

If L is additionally convex, then Adam-atan2 converges to the constrained minimizer,

$$x_{\infty} \in \arg \min_{\|x\|_{\infty} \leq 1/\lambda} L(x).$$

Proof. The proof relies structurally on the same arguments as Xie & Li (2024, Theorem 1.1). We prove that the limit point of Adam-atan2 satisfies the conditions of Lemma 2.2 by showing that $|\Delta_{\infty, i}| \leq 1$ for all components i , from which the Corollary follows.

We continue off at the point of analyzing the behavior of the asymptotic gradient $g_{\infty, i}$. Again for simplicity we fix and drop the index i . Let $u_t = m_t / \sqrt{v_t} \rightarrow u_{\infty}$ and $\Delta_t = \frac{4}{\pi} \arctan u_t$. Note that Theorem 2.1 bounds the second moment of what we denote here as u_t . If $g_{\infty} \neq 0$, then

$$\Delta_{\infty} = \frac{4}{\pi} \arctan u_{\infty} \rightarrow \frac{4}{\pi} \arctan \text{sign}(g_{\infty}) = \text{sign}(g_{\infty})$$

and so $|\Delta_{\infty}| = 1$. If $g_{\infty} = 0$, we endeavor to bound Δ_{∞} directly. Let $\varphi(w) = \frac{4}{\pi} \arctan \sqrt{w}$. It has second derivative

$$\varphi''(w) = -\frac{1 + 3w}{\pi w^{3/2}(1 + w)^2} < 0$$

for $w > 0$, so $\varphi(w)$ is concave in the same domain. By Jensen's inequality and the bound on the second moment of u_t in Theorem 2.1,

$$\begin{aligned} |\bar{\Delta}_T| &= \left| \frac{1}{S_T} \sum_{t \leq T} \eta_t \Delta_t \right| \leq \frac{1}{S_T} \sum_{t \leq T} \eta_t \varphi(u_t^2) \\ &\leq \varphi \left(\frac{1}{S_T} \sum_{t \leq T} \eta_t u_t^2 \right) \leq \varphi \left(1 + \frac{O(1)}{S_T} \right) = 1 + \frac{\varphi'(1) \cdot O(1)}{S_T} + O(S_T^{-2}). \end{aligned}$$

Taking the limit as $T \rightarrow \infty$ we conclude $|\bar{\Delta}_\infty| \leq 1$. The rest of the proof follows as in Theorem 2.1. $\square$

Thus, we have shown that the limit point of Adam-atan2 in the full-batch setting, if it converges, is a KKT point of the same ℓ_∞ -norm constrained optimization problem as that of AdamW.

2.4.3 SGD

We also analyze the behavior of models trained with Stochastic Gradient Descent (SGD), see Algorithm 6. In contrast to adaptive algorithms, SGD has an implicit bias for large ℓ_2 -margin separators in the linear setting.

Algorithm 6 Stochastic Gradient Descent (SGD) with Nesterov Momentum

Require: Learning schedule $\{\eta_t\}_{t \in T}$, weight decay λ , momentum factor $\beta \in [0, 1)$, initial parameters θ_0 , objective function $L(\theta)$

Ensure: Final parameters θ_T

```

1:  $v_0 \leftarrow 0$ 
2:  $t \leftarrow 0$ 
3: while  $\theta_t$  not converged do
4:    $t \leftarrow t + 1$ 
5:    $g_t \leftarrow \nabla_\theta L(\theta_{t-1}) + \lambda \theta_{t-1}$ 
6:    $v_t \leftarrow \beta v_{t-1} + g_t$ 
7:    $\tilde{g}_t \leftarrow g_t + \beta v_t$ 
8:    $\theta_t \leftarrow \theta_{t-1} - \eta_t \tilde{g}_t$ 
9: end while
10: return  $\theta_t$ 

```

In direct analogy with the KKT-characterization of limit points of adaptive momentum we prove the following stronger theorem:

Theorem 2.6. *Let $L : \mathbb{R}^d \rightarrow \mathbb{R}$ be a continuously differentiable objective function, $\beta \in [0, 1)$ the momentum parameter, λ the weight decay coefficient, $x_0, v_0 \in \mathbb{R}^d$ parameter initializations and $\{\eta_t\}_{t=0}^\infty$ (not necessarily non-increasing) learning schedule such that $\eta_t \geq 0$ and $\sum_t \eta_t = \infty$. Consider full-batch GD with weight decay and Nesterov (look-ahead) momentum*

$$\begin{aligned} v_{t+1} &= \beta v_t + \nabla L(x_t) + \lambda x_t \\ x_{t+1} &= x_t - \eta_t (\nabla L(x_t) + \lambda x_t + \beta v_{t+1}) \end{aligned}$$

If $x \rightarrow x_\infty$ converges under this algorithm, then

1. $\nabla L(x_\infty) + \lambda x_\infty = 0$ and x_∞ is a stationary point of $L(x) + \frac{\lambda}{2} \|x\|_2^2$.
2. With $R = \|x_\infty\|_2$, x_∞ is a KKT point of the ℓ_2 -norm constrained optimization problem

$$\min_{\|x\|_2 \leq R} L(x).$$

3. Further, if L is convex, $x_\infty \in \arg \min_{\|x\|_2 \leq R} L(x)$ is a global minimizer.

Note that the ℓ_2 -norm R of x_∞ is unbounded, in contrast to the ℓ_∞ case where the norm is bounded by $1/\lambda < \infty$.

Proof. 1. Define $c_t = \nabla L(x_t) + \lambda x_t$, and $u_t = c_t + \beta v_{t+1}$. Since ∇L is continuous, $c_t \rightarrow c_\infty = \nabla L(x_\infty) + \lambda x_\infty$ as $t \rightarrow \infty$. The recursion on momentum v_t says $v_{t+1} = \beta v_t + c_t$, so

$$v_{t+1} = \beta^{t+1} v_0 + \sum_{i=0}^t \beta^{t-i} c_i.$$

Using $c_\infty/(1-\beta) = \sum_{i \geq 0} \beta^i c_\infty = \sum_{i=0}^t \beta^i c_\infty + \sum_{i>t} \beta^i c_\infty$,

$$v_{t+1} - \frac{c_\infty}{1-\beta} = \beta^{t+1} v_0 + \sum_{i=0}^t \beta^{t-i} (c_i - c_\infty) - \frac{\beta^{t+1}}{1-\beta} c_\infty.$$

The first and last term vanish as $t \rightarrow \infty$ since $\beta < 1$. For the middle term, let $M = \sup_i \|c_i - c_\infty\| < \infty$ finite because $c_i \rightarrow c_\infty$. For every $\epsilon > 0$ let N be such that $\|c_i - c_\infty\| \leq \epsilon$ for all $i \geq N$. Then for $t \geq N$:

$$\left\| \sum_{i=0}^t \beta^{t-i} (c_i - c_\infty) \right\| \leq M \sum_{i < N} \beta^{t-i} + \epsilon \sum_{i=N}^t \beta^{t-i} \leq \frac{M\beta^{t-N+1}}{1-\beta} + \frac{\epsilon}{1-\beta}.$$

and the first term vanishes for sufficiently-large t . Therefore, $v_t \rightarrow c_\infty/(1-\beta)$. It follows that

$$u_t = c_t + \beta v_{t+1} \rightarrow c_\infty + \frac{\beta c_\infty}{1-\beta} = \frac{c_\infty}{1-\beta}.$$

Suppose for the sake of contradiction that $c_\infty = \nabla L(x_\infty) + \lambda x_\infty \neq 0$. First,

$$\langle c_\infty, u_t \rangle \rightarrow \frac{\|c_\infty\|_2^2}{1-\beta} > 0.$$

Thus, there exists s such that $\langle c_\infty, u_t \rangle \geq \|c_\infty\|_2^2/(2(1-\beta))$ for all $t \geq s$. Then

$$\langle c_\infty, x_0 - x_T \rangle = \sum_{t=0}^{T-1} \eta_t \langle c_\infty, u_t \rangle \geq \sum_{t=0}^{s-1} \eta_t \langle c_\infty, u_t \rangle + \frac{\|c_\infty\|_2^2}{2(1-\beta)} \sum_{i=s}^{T-1} \eta_i.$$

This quantity goes to ∞ as $T \rightarrow \infty$, contradicting the assumption that x_t converges to (finite) x_∞ . Therefore $c_\infty = 0$.

2. Let $R = \|x_\infty\|_2$ and

$$g(x) = \frac{1}{2} (\|x\|_2^2 - R^2).$$

The Lagrangian for the constrained optimization problem is given by

$$\mathcal{L}(x, \mu) = L(x) + \mu g(x).$$

Primal feasibility $g(x_\infty) \leq 0$ is true by definition of R . Stationarity $\nabla L(x_\infty) + \mu x_\infty = 0$ is true for dual variable $\mu^* = \lambda > 0$, which also satisfies dual feasibility. Complementary slackness $\mu g(x_\infty) = 0$ holds since $g(x_\infty) = 0$.

3. L and the constraint are convex, so KKT conditions are sufficient for a global minimum. $\square$

2.4.4 THE ROLE OF IMPLICIT BIAS

Fixing architecture and data, the implicit bias of optimization determines where in parameter space a neural network ends up after training to convergence. The above theoretical results in simplified (separable or full-batch) settings suggest that models trained with AdamW or Adam-atan2 should therefore be much more similar than those trained with SGD. Our results in Section 4.4 show that if similarity is measured by RSA on pre-activations, then this is indeed the case. Moreover, we track the RSA similarity throughout training, revealing interesting second-order information not present in the final checkpoint.

Another way to differentiate models trained with different optimizers is to consider “intra-optimizer” and “inter-optimizer” mode connectivity (see e.g. Zhang et al., 2026). Comparing models using RSA similarity therefore complements the functional similarity literature on the nature of generalization. It would be an interesting direction to study the connection of RSA with mode connectivity, e.g. by interpolating between models based only their pre-activation patterns.

2.5 DIMENSIONALITY REDUCTION FOR VISUALIZATION

Given a set $X = \{x_i\}_{i \in I}$ of trained models and pairwise “distances” $\mu_{ij} = \rho(x_i, x_j)$ between models as measured by representational similarity ρ , we can endeavor to reconstruct the space of models. We do this with two methods, (1) classical Multi-Dimensional Scaling (MDS) and (2) Isometric Mapping (Isomap), which itself builds on MDS.

The idea behind both methods is to optimize a loss function, the *strain*, which quantifies the disagreement between the input similarities between points and the realized/reconstructed distance. While MDS uses the similarity scores directly, Isomap additionally accounts for the manifold structure by using the geodesic distance induced by a neighborhood graph generated by the raw similarity scores. Schematically, the strain function for $\{x_i\}_{i \in I}$ is given by

$$\text{Strain}(x_1, \dots, x_n) = \left(\frac{\sum_{i,j} (\mu_{ij} - x_i^\top x_j)^2}{\sum_{i,j} \mu_{ij}^2} \right)^{1/2}$$

2.5.1 CLASSICAL MULTI-DIMENSIONAL SCALING (MDS)

Since CKA and PWCCA compute similarity scores, their complement $1 - \rho$ is a *dissimilarity* score rather than a distance (they do not satisfy the triangle inequality, and in the case of PWCCA, are not even symmetric). Therefore we use Non-metric Multi-Dimensional Scaling (NMDS), which aims to optimize the objective

$$S(x_1, \dots, x_n; f) = \left(\frac{\sum_{i < j} (f(d_{ij}) - \hat{d}_{ij})^2}{\sum_{i < j} \hat{d}_{ij}^2} \right)^{1/2}$$

Here f is a monotonically-increasing function. NMDS is optimized by iteratively optimizing the two objectives $\hat{f} = \arg \min_f S(x_1, \dots, x_n; f)$ and $\arg \min_{\{x_i\}_{i=1}^n} S(x_1, \dots, x_n; \hat{f})$.

2.5.2 ISOMAP

As compared to MDS, Isomap uses a more sophisticated algorithm to determine a good low-dimensional representation of the model space. Heuristically, Isomap does the following:

1. Uses pairwise dissimilarity to find the nearest neighbors of each point. In our case we used a k -NN criterion with $k = 3$.
2. Construct a neighborhood graph where two models are connected by an edge if they are a k -NN.
3. Compute the shortest path (“geodesic distance”) between any two nodes using e.g. Dijkstra’s algorithm.
4. Compute a lower-dimensional embedding using pairwise geodesic distances.

Thus, Isomap performs well on datapoints which are assumed to have manifold structure. This is known to be the case through studies of mode connectivity (Garipov et al., 2018; Draxler et al., 2018). Low-loss models form low-dimensional submanifolds with convexity properties and have typical size on the order of distance to initialization (Yunis et al., 2022). Note that as a graph-based method, Isomap-based visualization is not appropriate with asymmetric RSA measures like PWCCA, and one should instead use a symmetrized variant of the measure.

3 EXPERIMENTAL SETUP

3.1 MODEL ARCHITECTURE

All experiments were performed using a 22-layer VGG-style convolutional autoencoder, whose encoder has identical structure to that of Morcos et al. (2018) and kernel size of 3×3 and padding of 1. Downsampling is accomplished with convolutional layers `Conv2d` with stride of 2 at layers 3, 6, 9. The number of channels of the encoder is 64, 64, 128, 128, 128, 256, 256, 512, 512, 512. The convolutional decoder is an exact mirror of the encoder with `ConvTranspose2d` in place of `Conv2d`. This architecture is summarized in Figure 1.

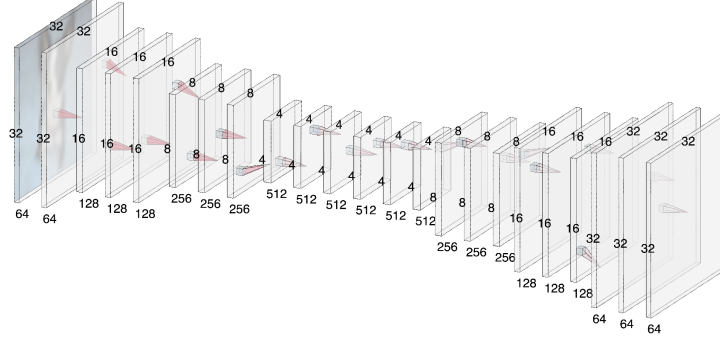


Figure 1: Architecture of the VGG-style convolutional autoencoder.

3.2 DATASET

Models were trained on the CIFAR-10 dataset with 50000 images split into 40000 images for training and 10000 images for validation and activation extraction. Image augmentation was not used, though in hindsight it may have been a good idea to do so to encourage rich feature learning. Comparing training and validation loss curves revealed that all models were able to generalize.

3.3 TRAINING PROTOCOL AND HYPERPARAMETERS

Models optimized with AdamW, Adam-atan2 were trained for 100 epochs with batch size of 32 with cosine-annealed learning rate and initial learning rate 0.001. The gradient moment hyperparameters were set to $\beta_1 = 0.9$, $\beta_2 = 0.999$ and a weight decay of 0.01 was used. Models optimized with SGD were trained for 200 epochs with batch size 256 again with cosine-annealed learning rate, but with initial learning rate of 0.005. Nesterov momentum was used, with friction coefficient 0.9, as well as weight decay of 0.0005.

Ten models per optimizer and parameterization were trained, distinguished by the random seed used for initialization and data-loading order. Theoretical and empirical results suggest that models sharing an optimizer and parameterization converge to the same mode after accounting for the permutation symmetry of neurons (Ainsworth et al., 2023). In the next section, we present visualizations enabled by MDS which exhibit this phenomenon.

4 RESULTS AND DISCUSSION

In this section we present and discuss the main results of the study.

4.1 SELF-SIMILARITY OF A MODEL DURING TRAINING

An easy way to organize RSA scores between models is to use a similarity matrix. The (i, j) -coordinates of a similarity matrix are the layers K_i of K and L_i of L , respectively, and the value of the cell at position (i, j) are $\rho(K_i, L_j)$, where ρ is either CKA or PWCCA. As a demonstration, we show in Figure 2 the similarity of a model’s layers during training versus the final checkpoint.

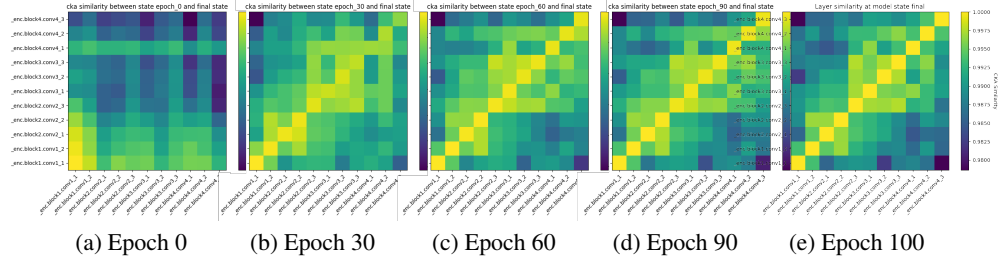


Figure 2: Layerwise CKA similarity of a model trained with Adam-atan2 at each epoch versus at the end of training.

The values on the diagonal of this matrix converge to unity because every layer of the model is exactly self-similar. It is also interesting to compare RSA similarities of layers of models trained with different optimizers (Figure 3).

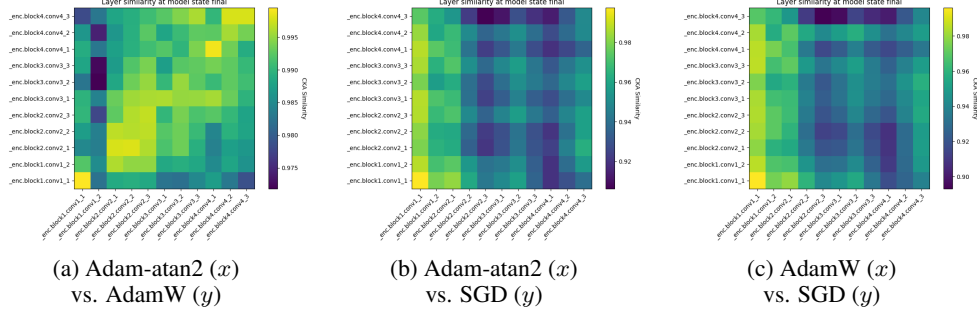


Figure 3: Layerwise CKA similarity between models trained using different optimizers.

4.2 OPTIMIZER IMPLICIT BIAS GUIDES FEATURE LEARNING

We plot the similarity between layers trained using different optimizers as a function of training epoch. To prevent the plot from being too cluttered, we only consider the last layer of each convolutional block: layer 2 (b1_out), layer 5 (b2_out), layer 8 (b3_out), and layer 11 (b4_out). In Figure 4 we present measurements using CKA similarity:

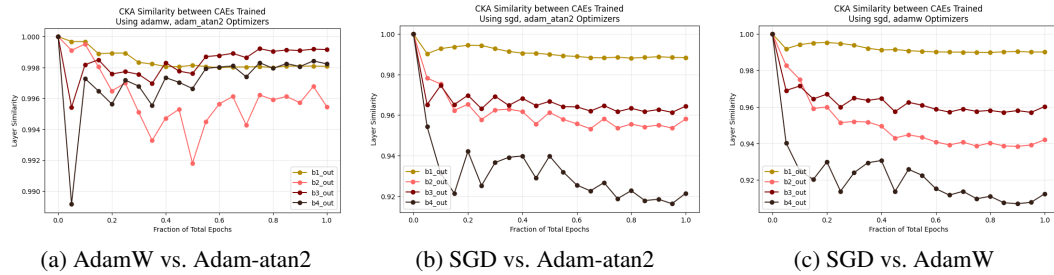


Figure 4: CKA similarity between layers of models trained with different optimizers as a function of training epoch.

Comparing models trained with SGD versus adaptive algorithms, we see that their RSA scores diverge over the course of training, with deeper layers drifting further than shallower ones. By contrast, the RSA score between representations learned with AdamW and Adam-atan2 do not display

a discernable pattern. We see a similar pattern when measuring similarity using PWCCA (Figure 5), though the plot shows much more variability in early stages of training.

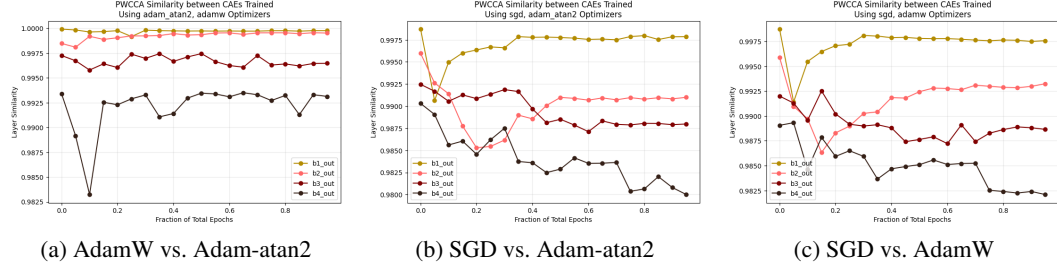


Figure 5: PWCCA similarity between layers of models trained with different optimizers as a function of training epoch.

Deep vision models learn a hierarchy of features: neurons in shallow layers are associated with low-level features while neurons in deeper layers are associated with higher-level ones. Thus, there are two possibilities that we can think of which explain our results:

1. Models trained with SGD and Adam rely on similar low-level features but on more different high-level features for the task of image compression and reconstruction.
2. Low-level features tend to be learned roughly at the same depth, while higher-level features can develop at a larger range in depth. From the viewpoint of information theory, neural networks train at the so-called “edge of chaos” (Bertschinger et al., 2004), a regime where information content is not lost (drowned out by irreducible noise) or amplified (leading to exploding gradients). In this regime the model is roughly scale-invariant (Schoenholz et al., 2017): layers $\ell, \ell + 1$ for $\ell \gg 1$ are roughly equivalent in context. This means that the same feature can be learned at variable depth depending on the initialization and randomization of data-loading. For shallower layers however, the approximate scale invariance is broken by finite-depth effects (Roberts et al., 2022), putting pressure on features to be learned at fixed depths.

The former explanation suggests a difference in learned features based on optimizer, while the latter suggests a difference in depth at which features are learned. Given that feature learning is largely data-dependent, we are biased towards the latter interpretation.

4.3 OPTIMIZATION AFFECTS HOW UNIFORMLY REPRESENTATIONS CONVERGE

We plot the RSA self-similarity scores of layers during training versus at their final state, for models trained with AdamW, Adam-atan2, and SGD (Figure 6).

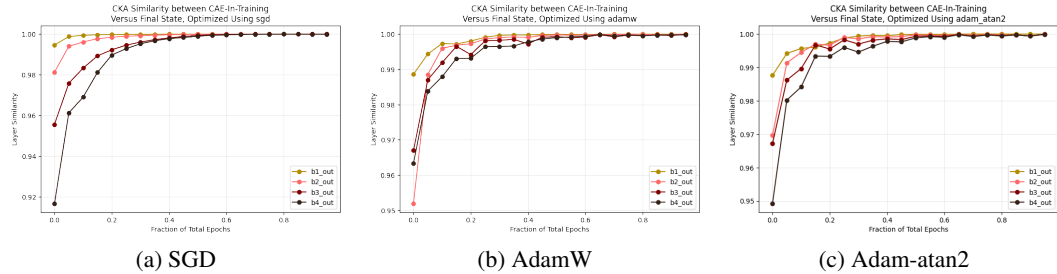


Figure 6: CKA self-similarity of layers during training versus their final state.

The results show a qualitative difference in the way layers converge to their final checkpoints depending on the optimizer used in training. Paying close attention to the difference in scale of the

y -axis, we see that the main difference between SGD and adaptive momentum is that the deeper layers are learned much faster for Adam, at a rate comparable to shallow layers. By contrast, weights optimized with SGD appear to be learned “bottom-up” with shallower layers converging to their final features at the Adam rate while deeper ones do so only after shallow layers have stabilized. This means that Adam enjoys better sample efficiency when training on this task of image compression and reconstruction. Combined with the results of Xie et al. (2025), this suggests that the loss landscape of the task at hand is characterized by ℓ_∞ -smoothness, rather than ℓ_2 -smoothness.

Now consider the same figures, but with similarities calculated by PWCCA (Figure 7).

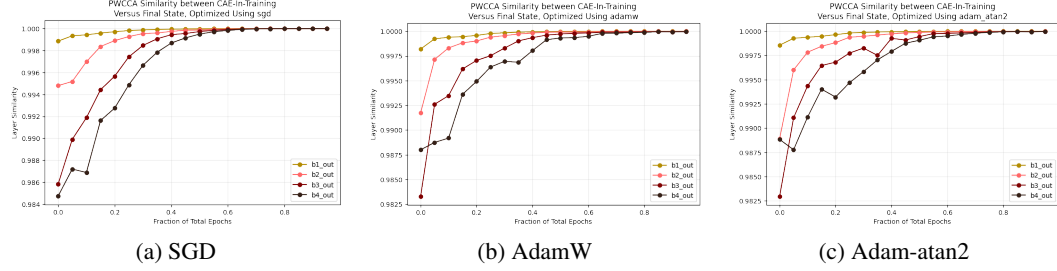


Figure 7: PWCCA self-similarity of layers during training versus their final state.

We see that the differences between SGD and adaptive methods disappear! So is the aforementioned phenomenon present or not? To answer this question, we performed a study to understand the sample efficiency of PWCCA and CKA. With fixed weights, we computed the PWCCA and CKA scores of AdamW versus SGD-optimized models using different number of samples m , i.e. so the activation vector takes the form $[z_1^{(\ell)}, z_2^{(\ell)}, \dots, z_m^{(\ell)}]$. We then plotted the scores as a function of m (Figure 8).

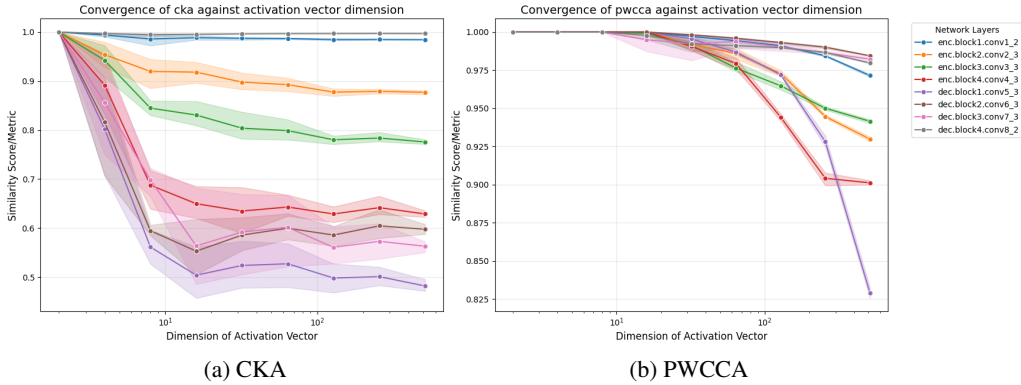


Figure 8: RSA scores between AdamW- and SGD-trained models as a function of the number of samples m . Bands are 95% CIs over three i.i.d. draws from the test set.

The pre-activation for each layer was computed for three different i.i.d. draws from the test set and the RSA scores were averaged over the results; the bands represent 95% CIs for each line. For both similarity measures, the variance of RSA scores decreased as a function of samples used to compute pre-activations. However, while CKA scores began stabilizing to their asymptotic values around $m = 16$ samples, PWCCA scores continued to increase beyond $m = 512$. More importantly, the rank-order of PWCCA similarity changes as a function of m , indicating that PWCCA is not a reliable measure of feature similarity in our use case. This gives credence to our initial observation that Adam does indeed have an advantage over SGD.

4.4 OPTIMIZATION AFFECTS THE FINAL LOCATION OF MODELS IN REPRESENTATION SPACE

For each optimizer $\in \{\text{SGD}, \text{AdamW}, \text{Adam-atan2}\}$, we train ten models with different random initializations and data-loading order. We then compute the (banded mean diagonal) aggregate dissimilarity scores (see Section 2.3) for every pair of models among the thirty models. We then optimized non-metric MDS and Isomap visualization algorithms with input the aggregate RSA-dissimilarity scores to obtain the low-dimensional representations of model space shown in Figure 9.

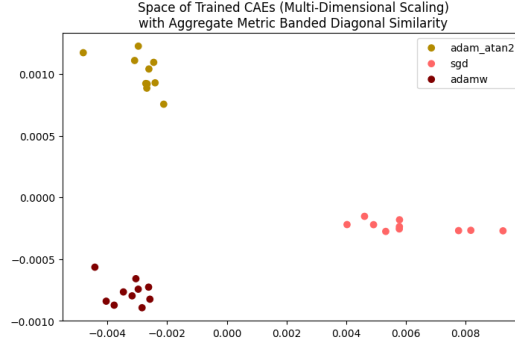


Figure 9: Non-metric MDS embedding of the thirty trained models using banded diagonal dissimilarity.

The n -dimensional coordinates $(x_1, \dots, x_n)$ of the embedding x of a point are ordered by their variance/importance from greatest to least. Therefore, MDS visualization shows that models trained with SGD are very distinct from those trained with adaptive momentum: the separation in the x -axis implies that the variance in feature dissimilarity can be best explained by the different optimizers.

The next most important direction distinguishes the two adaptive optimizers. Note however the much smaller separation between the populations. This suggests that there is finer, second-order differences in their implicit biases which steer models to different regions in the loss landscape, beyond the (identical) KKT-characterization of AdamW and Adam-atan2.

This conclusion is supported by plotting the aggregate similarity between models as a function of training epoch (Figure 10). The CKA similarity scores between the adaptive algorithms hover at a much higher value throughout training as compared to when compared with SGD.

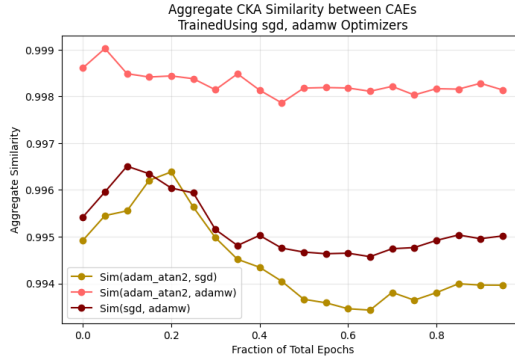


Figure 10: Aggregate CKA similarity between models as a function of training epoch.

The Isomap visualization (Figure 11) shows a different characterization of the loss landscape. The models trained with adaptive optimizers are no longer well-separated, though the models trained with adaptive optimizers and SGD can still be distinguished by a simple elliptical boundary.

The previous visualizations used banded diagonal dissimilarity (BDD) as the non-metric score for MDS and Isomap. We found that when using Mean Diagonal Dissimilarity (MDD) there were

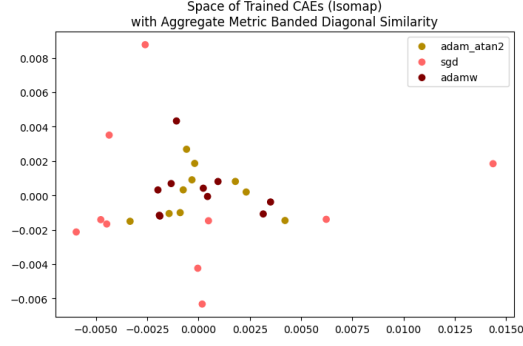


Figure 11: Isomap embedding of the thirty trained models using banded diagonal dissimilarity.

outlier points far from the main cluster (note the scale on the axes; Figure 12). We attribute these outlier points as having learned features in a layer which is offset as compared to most of the other models.

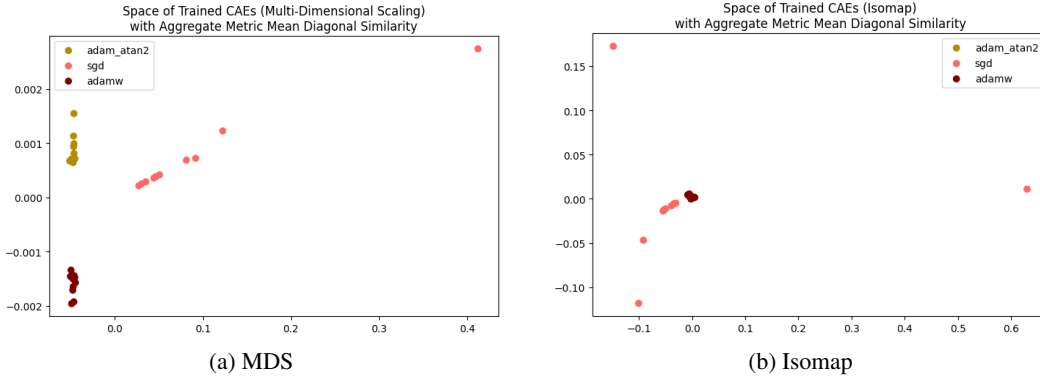


Figure 12: MDS and Isomap embeddings using mean diagonal dissimilarity (MDD).

4.5 LIMITATIONS

In this section we discuss limitations of our study. The first is the disagreement between the results of CKA and PWCCA. As discussed earlier in Section 4.3, PWCCA becomes unreliable at low number of samples. This is largely due to its algorithm: when $m < \max(d_1, d_2)$ the matrix K in the CCA algorithm is singular, with K gaining more zero eigenvalues as m decreases. This effectively cuts off the number of possible canonical directions and puts an upper bound on PWCCA score. To achieve a true empirical estimate of PWCCA similarity one must choose m to be larger than d_1, d_2 , but in this case the files containing activation data becomes prohibitively large (we use a moderate number $m = 128$ of samples). For this reason it is common nowadays to use other RSA measures such as CKA or Orthogonal Procrustes which can be robustly estimated with much fewer samples.

In estimating CKA, we have used the biased (V-statistic) estimator which is computationally light but biased for small sample sizes. This is related to the fact that terms like $\mathbb{E}k(X, X')$ in MMD^2 requires independent draws X, X' from the underlying distribution. The bias correction is of order m^{-2} for m samples and is negligible for our experiments.

Another limitation of our analysis is that we have not taken the finite mini-batch sizes into account. In hindsight, models should have been trained with full batch to get an accurate picture; mini-batches were used primarily for the goal of rich feature learning. Alternatively we could have investigated the implications of theoretical results in the mini-batch setting.

In our study we have also used an autoencoder with $O(10^7)$ parameters, much more than the 40,000 training samples of CIFAR-10. For classification tasks this would put us deep in the overfitting regime. Two competing forces work in our favor: (1) deep neural networks enjoy implicit regularization and they often happen to find good generalizing solutions despite being in this regime, (2) the trained task of image compression and reconstruction is much more difficult than a classification task: classifying CIFAR-10 involves picking one of 10 labels, while reconstructing a 32×32 RGB image involves predicting 3072 distinct output dimensions. It would be interesting to understand where exactly this places our setup in the double descent curve (Nakkiran et al., 2021).

Although PWCCA and CKA are commonly used in the RSA literature in both machine learning and computational neuroscience, they are not the only measures. A cross-disciplinary study (Klabunde et al., 2025) concluded that no single measure outperformed others across all benchmarks. A good choice of similarity measure depends on the symmetries of data and underlying model, as well as practical considerations of bias-variance tradeoff: an expensive algorithm cannot make too many comparisons. A common algorithm omitted in this study is Orthogonal/Permutation Procrustes, which schematically optimizes $\min_{g \in G} \|X - gY\|_2$ for G and orthogonal or permutation group. Another interesting, but less explored direction is to consider the covariance matrices $M = X^\top X$ of activation vectors X within the space of PSD matrices. A natural measure of similarity is the geodesic/earth mover’s distance (Shahbazi et al., 2021) on this space. Finally, neither PWCCA nor CKA satisfy the triangle inequality, a crucial property of metrics on normed vector spaces. A more correct approach, especially for Isomap visualization, would be to incorporate similarity measures which do (Williams et al., 2021).

5 SUMMARY AND FUTURE WORK

In this project we studied the effect of optimization on the training dynamics of deep neural networks trained on a real image dataset from the viewpoint of representational similarity analysis (RSA). We observe that optimizer implicit bias plays an important role in determining the final location of a trained model in the loss landscape, consistent with statistical learning theory. The novelty of our approach is using RSA analysis to arrive at this conclusion. To study this relationship, we first extended theoretical results on the implicit bias of optimizers to Adam-atan2 and batch GD with Nesterov momentum. We performed ablation studies by fixing the model architecture and dataset while varying initializations and data-loading orders across a fixed set of random seeds. Finally, we measured the similarity of trained models using the CKA and PWCCA scores.

It has been recently observed (Baek et al., 2026) that per-sample adaptive momentum optimizers, perhaps unexpectedly, have an implicit bias towards large ℓ_2 -margin separators on separable data instead of the expected ℓ_∞ -margin for models in the full-batch setting. If this is true, then what we assumed about the implicit bias of AdamW and Adam-atan2 may not apply here. To be sure, one must study the phase transition of implicit bias as a function of batch size. It would be interesting to understand how small-batch Adam relates to SGD, which has implicit bias for hard ℓ_2 -margin separators in linear settings. It would also be interesting to extend our analysis to include other optimization algorithms such as Muon (Jordan et al., 2024), Shampoo (Gupta et al., 2018), and Lion (Chen et al., 2024), all of which have many novel theoretical results characterizing their implicit biases (Fan et al., 2025; Wang et al., 2026; Sfyraki & Wang, 2025; Tsilivis et al., 2026).

Finally, it is interesting to wonder how our results, and more generally the science of RSA and interpretability, helps explain the nature of generalization in large language model pretraining and fine-tuning. The success of multi-modal models suggests a sort of vision-language convergence (Huh et al., 2024), though this position is hotly debated. We point out just one potential application for “representation engineering.” Just as RSA enables (Tung & Mori, 2019) the distillation of high-capacity teachers into more compute-efficient students, applying similar representation-matching techniques could help steer models towards safer and more useful behavior.

Another arena which benefits from a better understanding of rich feature learning is robotics and reinforcement learning. Because data is expensive to scale and environments often yield sparse rewards, a training pipeline with an inductive bias towards robotic goals could be much more data-efficient. Establishing an {architecture, data, optimizer}-independent characterization of feature learning could also help improve multi-sensor feature fusion, an essential component for Vision, Language, and Action (VLA) models like RT-2 (Brohan et al., 2023).

REFERENCES

- Samuel K. Ainsworth, Jonathan Hayase, and Siddhartha Srinivasa. Git re-basin: Merging models modulo permutation symmetries. In *International Conference on Learning Representations*, 2023. URL <https://arxiv.org/abs/2209.04836>.
- Guillaume Alain and Yoshua Bengio. Understanding intermediate layers using linear classifier probes, 2016. URL <https://arxiv.org/abs/1610.01644>.
- Beomhan Baek, Minhak Song, and Chulhee Yun. Implicit bias of per-sample Adam on separable data: Departure from the full-batch regime. In *International Conference on Learning Representations*, pp. 120180–120229, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/c3596a5990c2cddb0b0de478df7aca65-Abstract-Conference.html.
- Nils Bertschinger, Thomas Natschl ger, and Robert Legenstein. At the edge of chaos: Real-time computations and self-organized criticality in recurrent neural networks. In *Advances in Neural Information Processing Systems*, volume 17. MIT Press, 2004. URL <https://proceedings.neurips.cc/paper/2004/hash/f8da71e562ff44a2bc7edf3578c593da-Abstract.html>.
- Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control, 2023. URL <https://arxiv.org/abs/2307.15818>.
- Lizhang Chen, Bo Liu, Kaizhao Liang, and Qiang Liu. Lion secretly solves a constrained optimization: As Lyapunov predicts. In *International Conference on Learning Representations*, pp. 35404–35439, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/986e0caad271b59417287737416d8594-Abstract-Conference.html.
- Felix Draxler, Kambis Veschgini, Manfred Salmhofer, and Fred Hamprecht. Essentially no barriers in neural network energy landscape. In *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pp. 1309–1318. PMLR, 2018. URL <https://proceedings.mlr.press/v80/draxler18a.html>.
- Katie Everett, Lechao Xiao, Mitchell Wortsman, Alexander A. Alemi, Roman Novak, Peter J. Liu, Izzeddin Gur, Jascha Sohl-Dickstein, Leslie Pack Kaelbling, Jaehoon Lee, and Jeffrey Pennington. Scaling exponents across parameterizations and optimizers. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*. PMLR, 2024. URL <https://arxiv.org/abs/2407.05872>.
- Chen Fan, Mark Schmidt, and Christos Thrampoulidis. Implicit bias of spectral descent and Muon on multiclass separable data. In *Advances in Neural Information Processing Systems*, volume 38, pp. 39622–39669, 2025. doi: 10.52202/085713-1322. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/386432c7534eec9a1cd7cbeea90d7e9f-Abstract-Conference.html.
- Timur Garipov, Pavel Izmailov, Dmitrii Podoprikin, Dmitry Vetrov, and Andrew G. Wilson. Loss surfaces, mode connectivity, and fast ensembling of DNNs. In *Advances in Neural Information Processing Systems*, volume 31, 2018. URL https://proceedings.neurips.cc/paper_files/paper/2018/hash/be3087e74e9100d4bc4c6268cdeb8456-Abstract.html.
- Vineet Gupta, Tomer Koren, and Yoram Singer. Shampoo: Preconditioned stochastic tensor optimization. In *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*. PMLR, 2018. URL <https://arxiv.org/abs/1802.09568>.
- Jesse Hoogland. Neural networks generalize because of this one weird trick. Less-Wrong, 2023. URL <https://www.lesswrong.com/s/mqwA5FcL6SrHEQzox/p/fovfuFdpuEwQzJu2w>.

- Robert Huben, Hoagy Cunningham, Logan Smith, Aidan Ewart, and Lee Sharkey. Sparse autoencoders find highly interpretable features in language models. In *International Conference on Learning Representations*, pp. 7827–7845, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/1falab11f4bd5f94b2ec20e794dbfa3b-Abstract-Conference.html.
- Minyoung Huh, Brian Cheung, Tongzhou Wang, and Phillip Isola. The platonic representation hypothesis. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*. PMLR, 2024. URL <https://arxiv.org/abs/2405.07987>.
- Keller Jordan, Yuchen Jin, Vlado Boza, Jiacheng You, Franz Cesista, Laker Newhouse, and Jeremy Bernstein. Muon: An optimizer for hidden layers in neural networks, 2024. URL <https://kellerjordan.github.io/posts/muon/>.
- Max Klabunde, Tobias Schumacher, Markus Strohmaier, and Florian Lemmerich. Similarity of neural network models: A survey of functional and representational measures. *ACM Computing Surveys*, 57(9):1–52, 2025. doi: 10.1145/3728458.
- Simon Kornblith, Mohammad Norouzi, Honglak Lee, and Geoffrey Hinton. Similarity of neural network representations revisited. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pp. 3519–3529. PMLR, 2019. URL <https://proceedings.mlr.press/v97/kornblith19a.html>.
- Beren Millidge. Deep learning models are secretly (almost) linear. Blog post, 2023. URL <https://www.beren.io/2023-04-04-DL-models-are-secretly-linear/>.
- Ari Morcos, Maithra Raghu, and Samy Bengio. Insights on representational similarity in neural networks with canonical correlation. In *Advances in Neural Information Processing Systems*, volume 31, 2018. URL https://proceedings.neurips.cc/paper_files/paper/2018/hash/a7a3d70c6d17a73140918996d03c014f-Abstract.html.
- Preetum Nakkiran, Gal Kaplun, Yamini Bansal, Tristan Yang, Boaz Barak, and Ilya Sutskever. Deep double descent: Where bigger models and more data hurt. *Journal of Statistical Mechanics: Theory and Experiment*, 2021(12):124003, 2021. doi: 10.1088/1742-5468/ac3a74.
- Chris Olah, Alexander Mordvintsev, and Ludwig Schubert. Feature visualization. *Distill*, 2017. doi: 10.23915/distill.00007. URL <https://distill.pub/2017/feature-visualization>.
- Chris Olah, Nick Cammarata, Ludwig Schubert, Gabriel Goh, Michael Petrov, and Shan Carter. Zoom in: An introduction to circuits. *Distill*, 2020. doi: 10.23915/distill.00024.001. URL <https://distill.pub/2020/circuits/zoom-in>.
- Maithra Raghu, Justin Gilmer, Jason Yosinski, and Jascha Sohl-Dickstein. Svcca: Singular vector canonical correlation analysis for deep learning dynamics and interpretability. *Advances in neural information processing systems*, 30, 2017.
- Daniel A. Roberts, Sho Yaida, and Boris Hanin. *The Principles of Deep Learning Theory*. Cambridge University Press, 2022. URL <https://deeplearningtheory.com/>.
- Samuel S. Schoenholz, Justin Gilmer, Surya Ganguli, and Jascha Sohl-Dickstein. Deep information propagation. In *International Conference on Learning Representations*, 2017. URL <https://arxiv.org/abs/1611.01232>.
- Antonin Schrab. A practical introduction to kernel discrepancies: MMD, HSIC & KSD, 2025. URL <https://arxiv.org/abs/2503.04820>.
- Maria-Eleni Sfyraiki and Jun-Kun Wang. Lions and muons: Optimization via stochastic Frank-Wolfe under heavy-tailed noise, 2025. URL <https://arxiv.org/abs/2506.04192>.
- Mahdiyar Shahbazi, Ali Shirali, Hamid Aghajan, and Hamed Nili. Using distance on the Riemannian manifold to compare representations in brain and in models. *NeuroImage*, 239:118271, 2021. doi: 10.1016/j.neuroimage.2021.118271.

- Le Song, Alex Smola, Arthur Gretton, Justin Bedo, and Karsten Borgwardt. Feature selection via dependence maximization. *Journal of Machine Learning Research*, 13(47):1393–1434, 2012. URL <http://jmlr.org/papers/v13/song12a.html>.
- Daniel Soudry, Elad Hoffer, Mor Shpigel Nacson, Suriya Gunasekar, and Nathan Srebro. The implicit bias of gradient descent on separable data. *Journal of Machine Learning Research*, 19(70):1–57, 2018. URL <http://jmlr.org/papers/v19/18-188.html>.
- Adly Templeton, Tom Conerly, Jonathan Marcus, Jack Lindsey, Trenton Bricken, Brian Chen, Adam Pearce, Craig Citro, Emmanuel Ameisen, Andy Jones, Hoagy Cunningham, Nicholas L. Turner, Callum McDougall, Monte MacDiarmid, C. Daniel Freeman, Theodore R. Sumers, Edward Rees, Joshua Batson, Adam Jermy, Shan Carter, Chris Olah, and Tom Henighan. Scaling monosemanticity: Extracting interpretable features from Claude 3 Sonnet. *Transformer Circuits Thread*, 2024. URL <https://transformer-circuits.pub/2024/scaling-monosemanticity/index.html>.
- Nikolaos Tsilivis, Eitan Gronich, Julia Kempe, and Gal Vardi. Flavors of margin: Implicit bias of steepest descent in homogeneous neural networks. *Journal of Machine Learning Research*, 27(104):1–37, 2026. URL <http://jmlr.org/papers/v27/25-0634.html>.
- Frederick Tung and Greg Mori. Similarity-preserving knowledge distillation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, October 2019.
- Zixiao Wang, Yifei Shen, and Huishuai Zhang. OLion: Approaching the Hadamard ideal by intersecting spectral and ℓ_∞ implicit biases, 2026. URL <https://arxiv.org/abs/2602.01105>.
- Alex H. Williams, Erin Kunz, Simon Kornblith, and Scott Linderman. Generalized shape metrics on neural representations. In *Advances in Neural Information Processing Systems*, volume 34, pp. 4738–4750, 2021. URL https://proceedings.neurips.cc/paper_files/paper/2021/hash/252a3dbaeb32e7690242ad3b556e626b-Abstract.html.
- Shuo Xie and Zhiyuan Li. Implicit bias of AdamW: ℓ_∞ -norm constrained optimization. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 54488–54510. PMLR, 2024. URL <https://proceedings.mlr.press/v235/xie24e.html>.
- Shuo Xie, Mohamad Amin Mohamadi, and Zhiyuan Li. Adam exploits ℓ_∞ -geometry of loss landscape via coordinate-wise adaptivity. In *International Conference on Learning Representations*, pp. 7937–7965, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/1700ad4e6252e8f2955909f96367b34d-Abstract-Conference.html.
- David Yunis, Kumar Kshitij Patel, Pedro Henrique Pamplona Savarese, Gal Vardi, Jonathan Frankle, Matthew Walter, Karen Livescu, and Michael Maire. On convexity and linear mode connectivity in neural networks. In *NeurIPS 2022 Workshop on Optimization for Machine Learning (OPT)*, 2022. URL <https://openreview.net/forum?id=TZQ3PKL3fPr>.
- Chenyang Zhang, Difan Zou, and Yuan Cao. The implicit bias of Adam on separable data. In *Advances in Neural Information Processing Systems*, volume 37, pp. 23988–24021, 2024. doi: 10.52202/079017-0756. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/2ac79356a03fe5e9250e5e77ebc76e6e-Abstract-Conference.html.
- Fangzhao Zhang, Sungyoon Kim, Erica Zhang, Yiqi Jiang, and Mert Pilanci. Optimizer-induced mode connectivity: From AdamW to Muon, 2026. URL <https://arxiv.org/abs/2605.09991>.